Deferred Settlement on Cash Equities

The Open Window Between Trade-Date Recognition and Settlement-Date Custody Movement

Companion Document to the Ledger Framework Specification

R. Delloye

April 2026 — v1.0

Abstract

The Ledger framework [1] reduces post-trade activity to a single primitive: the atomic move of a quantity between wallets, governed by a conservation law that holds by construction. The framework is silent, however, on a fact peculiar to cash-equity markets: under the prevailing T+2 (and, for US equities since 28 May 2024, T+1) cycle, the economic event of buying or selling a share is separated from the custody event of receiving or delivering it by one or two business days. During that window the buyer is economically long without having received the security, and the seller is economically short of cash without having received it. The trading book, the risk system, and the regulator all need to know the position from T; the depot and the nostro do not move until $T + \text{ISD}$.

This document specifies how the Ledger represents that gap. The design is deliberately minimal. It adds two wallet classes (`cpty_virtual` and `csd_virtual`), one obligation row in the data layer's L_{15} leaf, and one workflow per obligation. It introduces *zero* new fields on `PositionState`, *no* fourth StatesHome map, *no* new conservation carve-out, and *no* new coordinate on the Generalised Position Model. The settlement window becomes a parameter of the standard machinery, not a doctrine of its own; T+0 atomic settlement on a distributed ledger and T+5 cross-border fail both fall out of the same construction. We build the design from scratch, demolish two naive alternatives along the way, walk through a standard buy and a standard sell at the level of individual moves with conservation verified at each step, extend the construction to seven variants and three composition cases, prove a Conservation Lifting Theorem that subsumes the open-window invariant, give a CDM crosswalk identifying five strategic gaps, and close with a list of nineteen invariants (DS1 through DS19) that an implementation must satisfy. The discipline of the document is pedagogical: a reader who stops after Section 4 has a working mental model; the remaining sections add complexity one variant at a time, each motivated by a case the simpler model could not handle.

Contents

1 The Problem

1.1 The economic fact and the custody fact diverge

Equity markets settle on a delay. When a portfolio manager places a buy order on the New York Stock Exchange and that order is matched at 14:32:11 UTC on Monday, the trade is *economically* done in the same instant: the manager has acquired exposure to the share's price, owes cash, and is entitled to dividends declared on or after that timestamp. The custody event — the actual transfer of the share into the buyer's depot account at the central securities depository (CSD) and the corresponding debit of the buyer's cash nostro — does not occur until Wednesday, the following day, or sometimes Thursday in cross-border cases. Between Monday and Wednesday the buyer sits in a state that no scalar wallet balance can cleanly describe: long the security in the eyes of the trading book, the risk system, the accounting standard, and the regulator; flat in the eyes of the CSD and the cash agent.

We will call this interval the *open settlement window*, denote it $[T, t_d]$ where T is trade-execution time and t_d is intended settlement date, and call its duration the *settlement cycle*. For US cash equities since 28 May 2024 the cycle is one business day (T+1); for EU and UK cash equities until at least 11 October 2027 it is two (T+2); for some Asian markets and atomic distributed-ledger settlement venues it is zero (T+0); for cross-border fails it can extend to five business days or more before regulatory buy-in is mandated. The duration is a parameter; the structural fact — that economic recognition and custody movement do not coincide — is universal.

1.2 A first concrete case

To make the divergence quantitative, consider a single trade, executed in clean conditions, with no failures or corrections.

- **Date of execution.** T = 2026-04-30, 14:32:11 UTC. The market is XNYS; the settlement cycle is T+2.
- **Intended settlement date.** t_d = 2026-05-04.
- **Trade.** Our portfolio (real wallet w_{us}) buys 100 shares of `XYZ` at $50.00 per share. The cash leg is $5,000.00; the counterparty is the broker Goldman Sachs (LEI `784F5XWPLTWKTBV3E584`, denoted hereafter GS).
- **Pre-trade state.** w_{us}.own(USD) = 1,000,000.00 and w_{us}.own(XYZ) = 0. The custodian's daily cash statement (an ISO 20022 `camt.053`) reports our nostro balance as $1,000,000.00.
- **Mark-to-market.** `XYZ` closes at $50.00 on Monday, $52.00 on Tuesday (T+1), and $51.50 on Wednesday (T+2).

The question is: *what does the Ledger know on Tuesday at the close of business?* On Monday no shares have arrived; no cash has left. On Tuesday no shares have arrived; no cash has left. The custodian's nostro statement still shows $1,000,000.00; the depot statement still shows zero `XYZ`. By any reading of the *custody* record, nothing has happened.

But our trading book has a position of +100 `XYZ` at T marked to $52.00 on Tuesday close, against a liability to pay $5,000.00 to GS on Wednesday. The portfolio's value has moved from $1,000,000.00 on Monday morning to:

$$V_{T+1} \;=\; 100 \cdot 52.00 \;+\; \underbrace{(1{,}000{,}000.00 - 5{,}000.00)}_{\text{cash net of payable}} \;=\; 5{,}200.00 + 995{,}000.00 \;=\; 1{,}000{,}200.00,$$

giving a profit-and-loss number of +$200.00 on Tuesday close, with *zero cash movement* and *zero custody movement*. This is the headline fact of trade-date accounting: the $200 is real, recognised under IFRS 9 paragraph B3.1.3 (the regular-way exception) and ASC 320-10-25-3, and it is reflected in the firm's regulatory capital, daily VaR, and overnight risk reports. It is

also a number that no system limited to scalar wallet balances over real custody accounts can produce, because there is nothing on those accounts that has changed.

1.3 What the Ledger must do

The Ledger has six obligations during the open window. The design of this document is the smallest extension that satisfies all six simultaneously.

1. **Recognise the economic position at T.** The portfolio is long 100 `XYZ` from 14:32:11 UTC on Monday. Mark-to-market PnL flows from T, not from t_d.
2. **Carry the per-counterparty obligation.** The buyer owes \$5,000.00 to GS; GS owes 100 `XYZ` to the buyer. These claims are net-settleable, ageing, capital-bearing, and counterparty-keyed. They must be observable as quantities, not just narratively.
3. **Reconcile to the external nostro and depot.** The custodian's `camt.053` and the CSD's depot statement remain authoritative for what is *at* the custodian and *at* the CSD. The Ledger must satisfy an algebraic identity with those statements at every wall-clock t, every business day, including during the open window.
4. **Register the legal duty.** A settlement obligation is a legal artefact governed by the master agreement, the CSD rulebook, and (in the EU) the CSDR fails regime. It has a deadline, a discharge predicate, a failure-handler, and a penalty clock. It must be a first-class object, not an attribute scattered across log lines.
5. **Discharge only on witness.** A move from *open* to *settled* is admitted only when an attested external observation arrives: a CSD finality message (`sese.025`), a custodian cash-credit advice (`camt.054`), or a multi-source quorum when those are silent. No clock-driven auto-discharge. No optimistic discharge. Silence is itself attested before any state change.
6. **Preserve replay determinism.** The witness substream is consumed in deterministic order; restated finality, partial credits, wire recalls, and out-of-order arrivals all converge to the same terminal state. This is the property that lets the Ledger time-travel and the property that lets a deterministic reference interpreter substitute for the production runtime in tests.

1.4 Document discipline

We will build the design move-by-move. After this section, we will demolish two strawmen (Section 2), define the state model (Section 3), walk through the canonical buy with conservation verified at every step (Section 4), do the same for the sell (Section 5), then add variants and composition cases (Section 6, Section 7), reconcile to the nostro (Section 8), state the accounting and regulatory footprint (Section 9), provide the CDM cross-walk (Section 10), give implementation notes (Section 11), declare what is out of scope (Section 13), and close with a glossary (Section 14). Throughout, every claim is concrete: every PnL number is computed in the document, every conservation table sums to zero in print, every invariant is stated in a numbered block, and every workflow is specified with its trigger, body, and termination condition.

2 Why the Naive Solutions Fail

Before describing the design, we describe two natural-looking alternatives and demonstrate that each violates one of the six requirements above. The exercise is not pedantic: each strawman embodies a misconception we have seen in production systems, and each demolition surfaces a structural requirement that the eventual design must satisfy.

2.1 Strawman 1: "Debit the nostro at T"

The proposal. On trade execution at T, atomically debit the cash nostro wallet by \$5,000.00 and credit the depot wallet by 100 `XYZ`. Treat the trade like an instantaneous settlement and reconcile to the custodian later.

The appeal. It is the simplest thing one can do on a single-primitive ledger: one move pair, one transaction, conservation holds atomically, no new state. The economic position is recognised at T (good); the per-counterparty obligation is implicit in the move (it was sourced from GS, registered in the metadata, fine); the legal duty becomes the move itself.

What breaks. The custodian's daily `camt.053` statement, sent at end of day Monday, will report a nostro balance of \$1,000,000.00. Our internal record will say \$995,000.00. The reconciliation fails. There are now two diverging stories about the same cash account:

- **Internal:** "We paid for the share. The cash is gone."
- **External:** "Your account holds \$1,000,000.00. Nothing has been debited."

The break is exactly \$5,000.00 every day for two days, with no operational meaning. This is not a transient anomaly; it is the system in normal working order. To resolve it, the operations team must invent a category called "cash in transit" and explain why it is sometimes a real liability (when the wire has been sent) and sometimes a fictional one (when it has not). Worse, on Wednesday morning when the nostro *is* actually debited by \$5,000.00 via the cash agent, the Ledger has nothing to apply that movement to: its internal state has already been reduced. Either the Wednesday `camt.054` cash-debit advice is treated as a no-op (in which case we have lost a piece of evidence the auditor will demand to see) or it is applied a second time (in which case \$5,000.00 disappears).

The deeper failure. The strawman *conflates two distinct facts*: the economic fact (that we now own the share from T) and the custody fact (that the cash actually leaves the nostro at t_d). They are different events, attested by different sources, on different business days, with different legal consequences. A ledger that collapses them loses information the framework cannot recover.

Requirement extracted. The economic move at T and the custody move at t_d must be *separately observable* and *separately attested*. They cannot share a single move-pair.

2.2 Strawman 2: "Recognise nothing until T+2"

The proposal. At T, record the trade as metadata only: a row in some pending-trades log. Make no moves. Wait until the CSD's `sese.025` arrives on Wednesday, then emit the move pair (depot +100 `XYZ`, nostro −\$5,000.00). The Ledger matches the external record at every instant.

The appeal. The ledger never disagrees with the custodian. Reconciliation is trivial. There is nothing to "unwind" on a fail; if Wednesday's message never arrives, no moves were ever made.

What breaks. On Tuesday close at \$52.00, the trading book asks the Ledger: "what is our P&L?" The Ledger responds: "You hold zero `XYZ`. Your cash is \$1,000,000.00. P&L is zero." This is wrong. The portfolio manager has \$200.00 of fair-value gain on the position they have already acquired. Under IFRS 9.B3.1.3 and ASC 320-10-25-3, a regular-way trade is recognised at trade date for fair-value-through-profit-or-loss instruments; settlement-date recognition is a banking-book exception that does *not* apply to a dealer trading book. The CRR Article 325 (FRTB) and Article 274 (SA-CCR) capital calculations *require* trade-date positions; a bank that uses settlement-date positions for its capital reports has misstated its capital.

The strawman is also operationally broken on the upstream side. The risk engine wants the position from 14:32:11 UTC on Monday for its overnight VaR; the trader's P&L blotter wants it for the daily mark; the regulatory transaction report (MiFIR Article 26 / RTS 22) is due at 23:59 Monday local time of the relevant national competent authority and must contain T, not t_d. None of these consumers can wait until Wednesday.

The deeper failure. The strawman *subordinates the economic record to the custody record.* The custody record is an attestation about a particular external book on a particular calendar; it is not the source of truth about who owns the position. The buyer owns the position from T regardless of what the CSD or the cash agent has done. A ledger that refuses to recognise that fact has chosen the wrong system of record.

Requirement extracted. The economic position must be *observable from* T on the trading book's wallet, by every consumer (risk, capital, regulatory, accounting), without waiting for or being conditional on the custody event.

2.3 The intersection of the two requirements

The two demolitions, taken together, give us the shape of the design.

- From Strawman 1: the cash and securities cannot *actually* move on the trading book's nostro and depot wallets at T; those wallets must remain consistent with the external records, which do not change until t_d.
- From Strawman 2: the economic position *must* be reflected on the trading book at T; there is no consumer for whom waiting is acceptable.

The only construction that satisfies both is one in which the trade-date move *recognises the economic position* (changes the trading book's view of what it owns) but the corresponding contra balance is held on a wallet that is *not* the external nostro or depot. The contra is per-counterparty, signed (because the same trade simultaneously creates a payable on cash and a receivable on the security), and explicitly distinct from the wallets that mirror external custody. The custody-date move at t_d then drains the contra into a separate wallet class that mirrors the external book at finality.

The next section makes this construction precise.

3 The State Model

The construction we settled on adds three things to the Ledger: two new *wallet classes*, one new *obligation row* in the existing data-layer leaf L_{15} (the data-spec `Obligation` leaf), and a *lifecycle finite-state machine* attached to that obligation row. It adds nothing to `PositionState`, nothing to `UnitStatus`, nothing to `ProductTerms`, no fourth StatesHome map, no new GPM coordinate, and no new conservation carve-out. The conservation law $\sum_{w \in \mathcal{W}} w_t(u) = 0$ holds over a constant universe $\mathcal{W}$ that grows by the new wallets but admits the new wallets as ordinary participants.

3.1 The triple

A deferred-settlement obligation is represented by three coordinated artefacts. The triple is the canonical mental model; everything else follows from it.

Definition 3.1 (Deferred-settlement triple)**.** *A trade τ that opens a settlement window of duration $t_d - T$ produces three coordinated artefacts on the Ledger:*

1. ***A real-wallet `own` write at T.*** *The trading book's real wallet w_{real} has its* own *coordinate updated by the standard v10.3 move algebra. This recognises the economic position. It is performed exactly once, at T, by the `apply_trade_move` handler. It is* never *reversed by any settlement event.*
2. ***A `cpty_virtual` contra balance.*** *For each $(w_{\text{real}}, cpty\_lei, u)$ triple, a single virtual wallet $w_{\text{PS}}[w_{\text{real}}, cpty, u]$ holds the per-counterparty open-obligation quantity. Its sign carries the side: positive means we owe; negative means they owe us. It is debited or credited at T atomically with the real-wallet move, and drained at t_d when finality is attested.*
3. ***An*** L_{15}.Obligation ***row.*** *A row in the data-layer's L_{15} leaf, kind `SettlementInstructionDelivery`, carrying the obligation's lifecycle state, intended settlement date, discharge predicate, CSDR clock, and audit chain. It is created at T, advanced by witness arrival, and reaches a terminal state at or after t_d.*

The division of labour is sharp. The `cpty_virtual` pair holds the *contra-quantity*; the L_{15} row holds the *lifecycle*. There is no transaction-level FSM in storage — the trade-level status (`Settled`, `InFlight`, `PartiallySettled`, ...) is a pure projection function over the per-leg L_{15} rows, computed on read. We define this projection in Section 3.7.

3.2 Wallet classes

Definition 3.2 (Wallet classes)**.** *Every wallet in $\mathcal{W}$ belongs to exactly one of four classes, recorded as a sidecar field on the* WalletRegistry*:*

`real`
: *A book-of-record wallet whose `own` coordinate represents an economic position the firm holds. Examples: w_{us} (the firm's portfolio), w_{book42} (a particular trading book). Real wallets are written at T by `apply_trade_move` and never moved by settlement.*

`cpty_virtual`
: *A per-counterparty virtual wallet keyed $(w_{\text{real}}, cpty\_lei, u)$, holding the open-window contra-quantity for that real wallet against that counterparty for that unit. Surface forms: $w_{\text{PS}}[w_{\text{real}}, \text{cpty}, \text{ccy}]$ for cash-leg obligations, $w_{\text{PSS}}[w_{\text{real}}, \text{cpty}, \text{ISIN}]$ for securities-leg obligations. Both surface forms are instances of the same wallet class.*

`csd_virtual`
: *A mirror wallet that represents the position of an external holder at a CSD or correspondent bank, keyed* $(\text{csd\_or\_bank\_lei}, \text{holder\_lei}, u)$. *Surface forms: $w_{\text{DTC\_depot\_mirror}}[holder]$,*

$w_{\text{JPMC\_nostro\_mirror}}[holder]$. *`csd_virtual` wallets absorb the contra at finality, mirroring what the external book records.*

`inception`
: *The singleton genesis wallet $w_{\text{genesis\_inception}}$ that establishes the absolute baseline of the constant universe per v10.3 §2.7's inception-move discipline. Every pre-trade real-wallet balance has a corresponding equal-and-opposite entry on $w_{\text{genesis\_inception}}$ so that the absolute form of conservation $\sum_{w \in \mathcal{W}} w_t(u) = 0$ holds from the first state. The inception wallet is written exactly once per unit at registry creation and is never moved thereafter; it is omitted from per-state delta tables in worked examples (because $\Delta = 0$) but is part of $\mathcal{W}$.*

Remark 3.3 (Side is sign, not class)**.** *Earlier drafts of this design carried a four-class taxonomy with `cpty_virtual_payable` and `cpty_virtual_receivable` as separate classes. We collapsed them: the* side *of an obligation (do we owe, or do they owe us?) is encoded as the sign of the stored balance, not as a wallet class. A positive balance on $w_{\text{PS}}[w_{\text{us}}, \text{GS}, \text{USD}]$ means we owe GS USD; a negative balance means GS owes us USD. Disclosure-layer gross numbers, when required by IFRS 7 paragraph 13C, are projections:*

$$\begin{aligned} \text{gross\_payable}[w, \text{cpty}, u] &= \max(0, w_{\text{PS}}[w, \text{cpty}, u].\text{own}), \\ \text{gross\_receivable}[w, \text{cpty}, u] &= \max(0, -w_{\text{PS}}[w, \text{cpty}, u].\text{own}). \end{aligned}$$

Storage is signed; disclosure is gross by projection. This eliminates a class enum that would otherwise leak through the type system.

3.3 Sidecar metadata on `WalletRegistry`

The new wallet classes induce sidecar columns on the `WalletRegistry`, none of which are state. They are KYC-style metadata that resolve at registry creation and do not vary with transactions:

```
WalletRegistry sidecar columns:
  wallet_class             : { real | cpty_virtual | csd_virtual | inception }
  real_wallet              : FK to the owning real wallet
                             (NULL for real wallets and csd_virtual)
  cpty_lei                 : LEI of the counterparty
                             (NULL if not counterparty-keyed)
  csd_lei                  : LEI of the CSD or correspondent bank
                             (NULL unless csd_virtual)
  holder_lei               : LEI of whose mirror this is
                             (NULL unless csd_virtual)
  expected_settle_window   : { T+0 | T+1 | T+2 | T+3 } per venue convention
```

The crucial point is that *no field is added to `PositionState`*. Both `cpty_virtual` and `csd_virtual` balances live in the same `PositionState[wallet, unit]` table that holds real-wallet balances, on the same `own` coordinate. The new wallets are ordinary rows in WalletRegistry; their balances are ordinary rows in PositionState. The framework's existing scan-by-wallet, project-on-read, conservation-by-construction machinery applies uniformly.

3.4 StatesHome compliance

The design is constrained by the v10.3 StatesHome principle: every piece of state lives in exactly one of the three maps (`ProductTerms`, `UnitStatus`, `PositionState`) plus the `WalletRegistry`. The deferred-settlement extension touches only PositionState (via new wallet rows on the existing own coordinate) and WalletRegistry (via the sidecar columns above). It does not add a fourth map. Table 1 records the explicit check.

The lifecycle FSM lives on the L_{15}.Obligation row, which is part of the data layer's smart-contract execution data, not StatesHome. This is the right home: L_{15} is exactly the leaf the data spec reserves for canonical obligation records.

StatesHome map	Touched?	How?
ProductTerms$[u]$	no	cash and security terms unchanged
UnitStatus$[u]$	no	instrument-level lifecycle unaffected
PositionState$[w, u]$	yes	only via existing handlers; only on `own`
WalletRegistry$[w]$	yes	sidecar metadata (3 classes)

Table 1: StatesHome conformance check.

3.5 The constant universe and conservation

Definition 3.4 (Constant universe)**.** $\mathcal{W} = \mathcal{W}_{\text{real}} \cup \mathcal{W}_{\text{cpty\_virtual}} \cup \mathcal{W}_{\text{csd\_virtual}} \cup \mathcal{W}_{\text{inception}}$ *is the* constant *wallet universe over which conservation is asserted. "Constant" here means: the same set of wallet identifiers at every wall-clock t; new wallets enter the universe at a defined zero balance for every unit.*

Invariant 3.5 (Conservation over $\mathcal{W}$)**.** *For every unit u and every wall-clock t,*

$$\sum_{w \in \mathcal{W}} w_t(u) \;=\; 0.$$

The sum runs over all real, cpty_ virtual, csd_ virtual, and inception wallets in the universe. The invariant admits no carve-out for state-only transactions (which contribute zero to every wallet sum) or for finality (whose move pair sums to zero). Section 8.8 proves this as a corollary of the Conservation Lifting Theorem with five named hypotheses.

3.6 The lifecycle FSM

The legal duty associated with each obligation row is a finite-state machine. We give the closed-sum specification first; the transition rules and witness discipline follow.

Definition 3.6 (Lifecycle states)**.** *The lifecycle of an* L_{15}.Obligation *row is described by a state $\sigma(o) \in \mathcal{L}$, where $\mathcal{L}$ is the closed sum:*

$\mathcal{L} = \{\texttt{Pending}, \texttt{Instructed}, \texttt{PartiallySettled}, \texttt{Discharged}, \texttt{Failed}(r), \texttt{BoughtIn}, \texttt{Cancelled}, \texttt{Compens}$

where r ranges over a closed-sum ***`FailureReason`****. The terminal set is* $F_{\text{terminal}} = \{\texttt{Discharged}, \texttt{Cancelled}, \texttt{Boug}$
`Failed` *is terminal-conditional: it admits transitions to* ***`BoughtIn`*** *or* ***`Compensated`*** *via the CSDR resolution path; otherwise it is absorbing.*

```
type LifecycleState =
  | Pending             (* registered at T; awaiting instruction emission   *)
  | Instructed          (* sese.023 dispatched; awaiting CSD finality       *)
  | PartiallySettled    (* partial discharge witnessed; residual still open *)
  | Discharged          (* terminal: full witness-driven discharge          *)
  | Failed of FailureReason
                        (* terminal-conditional: deadline missed or fail
                           message; admits BuyIn or Compensated             *)
  | BoughtIn            (* terminal: CSDR mandatory buy-in resolved         *)
  | Cancelled           (* terminal: pre-instruction four-eyes CORRECTION   *)
  | Compensated         (* terminal: cash-in-lieu, close-out, manual override *)

type FailureReason =
  | DeadlineMissed
  | NoCover                       (* securities short for delivery   *)
  | NoFunds                       (* cash short for payment          *)
```

```
| CounterpartyDefault of LEI
| CsdReject of CsdRejectCode    (* normalised closed sum          *)
| LegInconsistent of WhichLeg   (* DvP partial-fail asymmetry     *)
| Manual of OperatorId
```

The allowed transitions are:

$$\begin{aligned}
\texttt{Pending} &\to \texttt{Instructed} \quad (\texttt{sese.023}\text{ dispatch witness})\\
\texttt{Pending} &\to \texttt{Cancelled} \quad (\text{four-eyes pre-instruction CORRECTION})\\
\texttt{Instructed} &\to \texttt{PartiallySettled} \mid \texttt{Discharged} \mid \texttt{Failed}(r) \quad (\text{witness})\\
\texttt{PartiallySettled} &\to \texttt{Discharged} \mid \texttt{Failed}(r) \quad (\text{witness on residual})\\
\texttt{Failed}(r) &\to \texttt{BoughtIn} \mid \texttt{Compensated} \quad (\text{CSDR resolution path})\\
\sigma &\to \texttt{Compensated} \quad (\text{four-eyes manual override; } \sigma \notin F_{\text{terminal}}).
\end{aligned}$$

All other transitions are forbidden by the closed-sum constraint. Compile-time enforcement is via exhaustive pattern match on `LifecycleState` in the step function (Section 11). Figure 1 draws the transitions.

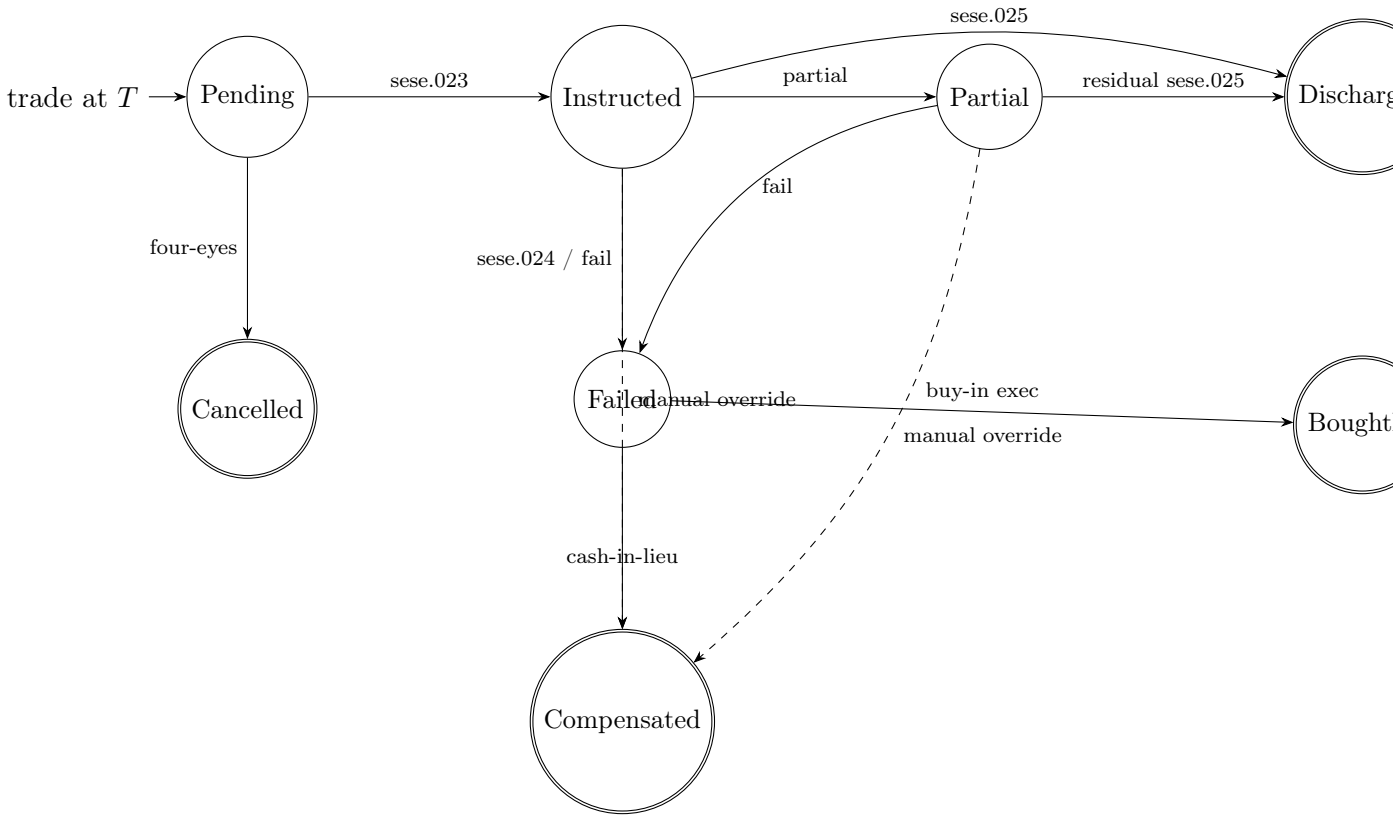

Figure 1: The lifecycle FSM. Solid arrows are witness-driven transitions; dashed arrows are four-eyes manual overrides. Accepting states are the terminal set $F_{\text{terminal}} = \{\texttt{Discharged}, \texttt{BoughtIn}, \texttt{Cancelled}, \texttt{Compensated}\}$. `Failed` is terminal-conditional.

3.7 Transaction-level status as a projection

A typical DvP cash-equity trade has *two* obligation rows: one for the securities leg (o_{sec}) and one for the cash leg (o_{cash}). They are independent in the FSM; each is transitioned by its own witness. There is, however, a useful trade-level question: "has this trade settled in full?" The Ledger answers it not with a stored field but with a pure projection function over the leg states.

```
def tx_status(tx: Transaction) -> TransactionView:
    legs = [o for o in L_15 if o.tx_id == tx.tx_id]
    states = {o.state for o in legs}
    if all(s == Discharged for s in states):                              return
    Settled
    if any(s == Compensated for s in states) and rest_terminal:           return
    Compensated
    if any(s == Cancelled   for s in states) and rest_terminal:           return
    Cancelled
    if any(s == PartiallySettled for s in states):                        return
    PartiallySettled
    if any(s == Failed for s in states) and rest_terminal:                return
    Failed
    if all(s == Instructed for s in states):                              return
    Instructed
    if all(s in {Pending, Instructed} for s in states):                   return
    InFlight
    return Mixed   # leg-inconsistent; spawn wf-confirm-break

class TransactionView = Settled | Compensated | Cancelled | PartiallySettled
                      | Failed | Instructed | InFlight | Mixed
```

The function is pure, total over the input domain, and `Mixed` is the well-defined return for a leg-inconsistent state (one leg `Discharged`, the other `Failed`) which spawns a `wf-confirm-break` workflow. Operational queues that need to filter on transaction-level status use a materialised view computed by the same projection at write time and indexed; the eventual-consistency window is sub-second.

The reason `tx_status` is a projection rather than a stored field is structural. Storing it would create a second source of truth for trade-level state, vulnerable to the very class of reconciliation failure the framework is designed to eliminate. Computing it forces the per-leg states to remain canonical and the trade-level view to remain dependent — which is the architecturally correct relationship.

3.8 Witness-driven discharge

The single most important rule of the lifecycle is that no transition out of `Pending` (other than four-eyes `Cancelled`) occurs without a verified witness.

Principle 3.7 (Witness-driven discharge)**.** *An obligation o transitions to a non-`Pending`, non-`Cancelled` state if and only if:*

1. *An* attestation envelope *(defined in Section 11.1) has been recorded on the data layer's external-confirmation leaf L_{11} (the data-spec `ExternalConfirmation` leaf);*
2. *The envelope's signature, schema, and dedup-key have been verified, and (where required by the multi-source policy) the envelope is corroborated by a quorum of independent sources;*
3. *The envelope's payload reconciles with the obligation's discharge predicate within the per-currency tolerance pinned in L_7^{P} (the data-spec `PolicyConfiguration` sidecar).*

The framework never *marks a leg `Discharged` by inference, by clock, or by absence of evidence. Watchdog timers trigger a break workflow, not an auto-fail (Section 11.3).*

The principle is non-trivial. A naive workflow would say: "if no fail message has arrived by $t_d + 0.5$ bd, mark the obligation `Discharged` optimistically." This is wrong on two counts: it admits silence as evidence, and it makes the ledger's state a function of clocks rather than of evidence. The framework's discipline is that silence is itself attested — a `silence_attestation` envelope is generated by the watchdog, recorded on L_{11}, and used to open a break workflow that escalates for human action — but the underlying obligation *remains* in `Pending` or `Instructed` until a positive witness arrives or a four-eyes correction transitions it explicitly. This is what

makes the ledger time-travellable: the state at any historical instant is exactly the state implied by the witnesses arrived by that instant, never the state that some clock or default might have produced.

3.9 What we did not add

It is worth listing the alternatives we considered and rejected, so the design decisions are visible.

- **A seventh GPM coordinate (`inflight`).** Adding a per-(wallet, unit) `inflight` field to `PositionState` would make the open-window quantity directly visible. Rejected: it puts settlement-FSM state on `PositionState`, breaks the StatesHome single-coordinate principle, and confuses `own` with inflight. The same information is available as an indexed projection over L_{15}; storage and projection are the right separation.
- **First-class unit u°.** Instead of cpty_virtual wallets, every obligation could fabricate a fresh unit u°_o keyed by the obligation, and the contra balances would live in PositionState$[w_{\text{real}}, u^\circ_o]$. Section 10.5 proves this is information-equivalent (a bijection Φ). Rejected on operational grounds: it would inflate the unit master from $\mathrm{O}(10^4)$ to $\mathrm{O}(10^7)$, and it forces every consumer of PositionState to opt out of obligation-units. Mainstream is chosen on cost; the bijection means there is no formal-mathematical reason to prefer one.
- **Single-aggregate-per-CSD wallet.** Maintaining one $w_{\text{DTC\_aggregate}}$ that records the net position at DTC across all counterparties. Rejected: it loses the per-counterparty resolution required by IFRS 7 disclosure, the CSDR penalty matrix, and the ECL-by-counterparty calculation under IFRS 9.
- **Settlement state purely external.** Treating L_{15} as a view over the CSD's own settlement record. Rejected: the legal duty of the firm exists between T and the CSD's record of finality, and during that interval the firm bears the obligation regardless of whether the CSD has booked anything. The internal record must own the lifecycle.

The state model is now complete. The next two sections walk it through, move by move, on the standard buy and the standard sell.

4 The Standard Buy, in Full

This section is the load-bearing concrete artefact of the document. We walk a single, ordinary T+2 cash-equity buy from execution at T through finality at $T + 2$, showing every move, every wallet balance, every conservation table, and every PnL number. Subsequent sections refer back here. A reader who stops after this section has a complete working mental model of the framework's deferred-settlement design.

4.1 Setup

The example matches Section 1: buy 100 `XYZ` at $50.00, T+2.

- **Trade.** Buy 100 `XYZ` at $50.00 per share; cash leg $5,000.00.
- **Execution time.** T = 2026-04-30 at 14:32:11 UTC.
- **Intended settlement.** t_d = 2026-05-04 (T+2).
- **Real wallet.** w_{us} (our portfolio).
- **Counterparty.** Goldman Sachs broker, LEI `784F5XWPLTWKTBV3E584`, denoted GS.
- **CSD and correspondent.** DTC for the securities depot, JPMC for the cash nostro. Two mirror wallets are needed: $w_{\text{DTC\_depot\_mirror}}[\text{GS}]$ (our internal mirror of GS's position at DTC) and $w_{\text{JPMC\_nostro\_mirror}}[\text{GS}]$ (our internal mirror of GS's nostro at JPMC). Both are class `csd_virtual`; both are flat at pre-trade.
- **Pre-trade balances.**

$$w_{\text{us}}.\text{own}(\text{USD}) = 1{,}000{,}000.00$$
$$w_{\text{us}}.\text{own}(\text{XYZ}) = 0$$

- **External nostro balance.** The custodian's daily `camt.053` reports $w_{\text{JPMC\_nostro}}.\text{own}(\text{USD}) = 1{,}000{,}000.00$ for our account. (This is an external book; we represent it via the mirror plus our own balance, never store the custodian's record directly.)
- **Mark-to-market.** `XYZ` closes at $50.00 on T, $52.00 on $T + 1$, $51.50 on $T + 2$.
- **Decimal discipline.** Prices are typed D_8 (`Decimal(18,8)`); whole-share securities D_0; major cash D_2; *no floats*, `ROUND_HALF_EVEN`, no implicit FX. Per-currency tolerances are pinned in L_7^{P}.PolicyConfiguration (default $0.00 for major currencies).

We will track six wallets across the example, summarised in Table 2.

Wallet	Class	Role
w_{us}	real	our portfolio
$w_{\text{PS}}[w_{\text{us}}, \text{GS}, \text{XYZ}]$	cpty_virtual	per-cpty securities-leg contra
$w_{\text{PS}}[w_{\text{us}}, \text{GS}, \text{USD}]$	cpty_virtual	per-cpty cash-leg contra
$w_{\text{DTC\_depot\_mirror}}[\text{GS}]$	csd_virtual	mirror of GS's DTC depot
$w_{\text{JPMC\_nostro\_mirror}}[\text{GS}]$	csd_virtual	mirror of GS's JPMC nostro
$w_{\text{genesis\_inception}}$	inception	v10.3 genesis-move contra (kept implicit)

Table 2: The constant universe for the standard-buy example. The genesis-inception wallet establishes the absolute baseline per v10.3 §2.7 and is omitted from the per-state delta tables; it is what makes the absolute form of conservation hold for the pre-trade balance $1,000,000.00 on w_{us}.

4.2 The move block at T

At 14:32:11 UTC the execution report from the venue arrives at the executor. The Ledger emits exactly one transaction — type `ECONOMIC_RECOGNITION` (a sub-class of `SETTLEMENT`) — containing a move pair plus two state-only L_{15} registrations.

```
Transaction TX1:
  type:               ECONOMIC_RECOGNITION
  tx_id:
    hash_jcs(business_event_id="exec:XNYS:exec_id_42:leg_combined",
                               attempt_seq=0)
                    = "0xa7c3f1e8..."   (canonical formula in Section 11.2)
  ts:                 2026-04-30T14:32:11Z
  metadata:           { expected_settlement_date : 2026-05-04,
                        cpty_lei                 : GS_LEI,
                        venue_mic                : XNYS,
                        settlement_type          : DVP,
                        cdm_event                : ExecutionEvent }

  -- Securities leg: economic recognition. We become long 100 XYZ at T.
  Move 1:
    from = w_PS[w_us, GS, XYZ]      -- cpty_virtual; sign convention: +ve = we owe
    to   = w_us
    unit = XYZ
    qty  = D_0(100)

  -- Cash leg: economic recognition. We owe 5,000 USD at T.
  Move 2:
    from = w_us
    to   = w_PS[w_us, GS, USD]      -- cpty_virtual
    unit = USD
    qty  = D_2(5000.00)

  -- Atomically with the move pair, register the two obligations in L_15.
  L_15 register o_sec:
    obligation_id            : hash_jcs(tx_id, "leg_securities")
    kind                     : SettlementInstructionDelivery
    intended_settlement_date : 2026-05-04
    deadline                 : 2026-05-04T23:59:59Z (CSD timezone)
    discharge_predicate      : ByAttestation(Envelope, sese.025, ref=tx_id)
    compensation_handler     : SettlementFailHandler
    state                    : Pending

  L_15 register o_cash:
    obligation_id            : hash_jcs(tx_id, "leg_cash")
    kind                     : SettlementInstructionDelivery
    intended_settlement_date : 2026-05-04
    discharge_predicate      : ByAttestation(Envelope, camt.054, ref=tx_id)
    compensation_handler     : SettlementFailHandler
    state                    : Pending
```

Reading the moves

Move 1 sources 100 `XYZ` from $w_{\text{PS}}[w_{\text{us}}, \text{GS}, \text{XYZ}]$ and credits w_{us}. Before the move, the cpty_virtual wallet is at zero. After the move, it is at -100. By the sign convention, a negative balance means *they owe us*: GS owes us 100 `XYZ`. The credit on w_{us} records that the trading book is now long 100 shares.

Move 2 sources \$5,000.00 from w_{us} and credits $w_{\text{PS}}[w_{\text{us}}, \text{GS}, \text{USD}]$. Before, the cpty_virtual wallet was at zero. After, it is at +5,000.00. By the sign convention, a positive balance means *we owe*: we owe GS \$5,000.00.

The two cpty_virtual balances sit on opposite sides — one we owe, one they owe us — because in DvP the buyer's claim on the security and obligation on the cash are simultaneous

and reciprocal.

Conservation check at T

The atomic check on the move pair sums the deltas per unit across the constant universe. Table 3 shows the result.

Unit	Δw_{us}	Δw_{PS}[XYZ]	Δw_{PS}[USD]	Δ csd_virtual mirrors	Σ
XYZ	+100	−100	0	0	0
USD	−5,000.00	0	+5,000.00	0	0

Table 3: Conservation at T: per-unit sum of deltas across the constant universe is zero.

The sum is zero on both rows. Conservation holds, atomically, on the move pair.

Wallet snapshot post-TX1

```
w_us.own(XYZ)                          = +100
w_us.own(USD)                          = +995,000.00
w_PS[w_us, GS, XYZ].own(XYZ)           = -100          (GS owes us 100 XYZ)
w_PS[w_us, GS, USD].own(USD)           = +5,000.00     (we owe GS 5,000 USD)
w_DTC_depot_mirror[GS].own(XYZ)        = 0             (no finality yet; mirror flat)
w_JPMC_nostro_mirror[GS].own(USD)      = 0             (no finality yet; mirror flat)
```

4.3 At $T+1$ close

Tuesday closes at 21:00 UTC with XYZ at $52.00. *No moves are emitted on real wallets, no moves on cpty_virtual wallets, no moves on csd_virtual mirrors.* The position is unchanged. The settlement workflow may emit a state-only transaction TX1_INSTRUCT that contains zero moves and updates o_{sec}.state and o_{cash}.state from Pending to Instructed after dispatching the sese.023 settlement instruction to the CSD; conservation holds vacuously over zero moves.

PnL at $T+1$ close

Portfolio valuation derives directly from the State Model of Section 3: V_t is the sum, over the firm's *real* wallets, of qty · price. The cpty_virtual wallets (the w_{PS} pair) and the csd_virtual mirrors are accounting contras that exist solely to make conservation hold over the constant universe; they do *not* enter portfolio valuation. Concretely, $w_{\text{PS}}[w_{\text{us}}, \text{GS}, \text{USD}]$ holds +5,000.00, which is the firm's record of the cash that has *already left* w_{us} at T (note w_{us}.own(USD) went from $1,000,000 to $995,000); counting it again as a liability would double-count the same $5,000. Symmetrically, $w_{\text{PS}}[w_{\text{us}}, \text{GS}, \text{XYZ}] = -100$ is the contra to w_{us}.own(XYZ) = +100; the economic position lives in w_{us}, the contra exists only for conservation.

The canonical formula is therefore:

$$\boxed{V_t \;=\; w_{\text{us}}.\text{own}(\text{USD}) \;+\; w_{\text{us}}.\text{own}(\text{XYZ}) \cdot P_t(\text{XYZ})}$$

Substituting the four time-points:

$$\begin{aligned} V_{\text{pre}} &= 1{,}000{,}000.00 + 0 \cdot 50.00 &&= 1{,}000{,}000.00, \\ V_T &= 995{,}000.00 + 100 \cdot 50.00 &&= 1{,}000{,}000.00, \\ V_{T+1} &= 995{,}000.00 + 100 \cdot 52.00 &&= 1{,}000{,}200.00, \\ V_{T+2} &= 995{,}000.00 + 100 \cdot 51.50 &&= 1{,}000{,}150.00. \end{aligned}$$

$V_T = V_{\text{pre}}$ confirms what the trade is: a fair exchange at T at the prevailing price, zero PnL at the trade instant. The mark-to-market P&L is then:

$$\text{PnL}_{T+1} \;=\; V_{T+1} - V_T \;=\; +\$200.00, \qquad \text{PnL}_{\text{cum},T+2} \;=\; V_{T+2} - V_T \;=\; +\$150.00.$$

For completeness, the cpty_virtual balances are not lost; they reconcile via the recon identity of Section 8.1, not via valuation. $w_{\text{PS}}[w_{\text{us}}, \text{GS}, \text{USD}] = +5{,}000.00$ is the open-window cash payable that will drain to the JPMC nostro mirror at finality; $w_{\text{PS}}[w_{\text{us}}, \text{GS}, \text{XYZ}] = -100$ is the conservation contra to the position now booked in w_{us}. Both balances are visible to the disclosure layer (gross IFRS 7 projection, Herstatt scan) but contribute zero to V_t.

PnL at $T+1$ close: the headline

$$\text{PnL}_{T+1} \;=\; V_{T+1} - V_T \;=\; 1{,}000{,}200.00 - 1{,}000{,}000.00 \;=\; +\$200.00.$$

+\$200.00 **of P&L with zero cash movement and zero settlement movement.** This is the headline guarantee of trade-date economic recognition: the position is real, the mark is real, the P&L is real, and the custody fact has not yet occurred. This is precisely what Strawman 2 (Section 2.2) could not produce.

4.4 At $T+2$ morning, before finality

Wednesday morning, before any `sese.025` or `camt.054` arrives. No moves. The reconciliation engine runs the morning recon scan (Section 8). The recon identity holds at this instant; we will verify it explicitly in Section 8.3.

4.5 At $T+2$ finality

DTC reports DvP finality. Two messages arrive on the inbound feed and are written to the data layer's external-confirmation leaf L_{11} wrapped in attestation envelopes:

- An ISO 20022 `sese.025` (settlement confirmation) attesting that 100 `XYZ` have been transferred at DTC.
- An ISO 20022 `camt.054` (debit advice) attesting that \$5,000.00 has been debited at JPMC.

The `SettlementSaga` workflow consumes the signal pair (its commutativity behaviour is specified in Section 11.5; for the standard happy path the order does not matter) and emits the *finality contra-transaction*.

```
Transaction TX1_FINAL:
  type:    SETTLEMENT_FINALITY
  tx_id:   hash_jcs("FINAL", TX1.tx_id, sese.025_msg_id, camt.054_msg_id)
  ts:      2026-05-04T16:30:42Z

  -- Securities leg: drain the cpty_virtual receivable into the DTC depot mirror.
  -- The mirror was 0; it now records that GS has delivered 100 XYZ out of its DTC
  -- depot to us. From OUR books, GS's depot mirror goes short by 100.
  Move 1:
    from = w_DTC_depot_mirror[GS]
    to   = w_PS[w_us, GS, XYZ]
    unit = XYZ
    qty  = D_0(100)

  -- Cash leg: drain the cpty_virtual payable into the JPMC nostro mirror.
  -- The mirror records that GS has received 5,000 USD from us at JPMC.
  Move 2:
    from = w_PS[w_us, GS, USD]
    to   = w_JPMC_nostro_mirror[GS]
```

```
  unit = USD
  qty  = D_2(5000.00)

L_15 update o_sec.state:  Instructed -> Discharged
L_15 update o_sec.csd_finality_witness  : <sese.025 envelope dedup_key>
L_15 update o_cash.state: Instructed -> Discharged
L_15 update o_cash.csd_finality_witness : <camt.054 envelope dedup_key>
```

Reading the finality moves

The post-finality balances on the cpty_virtual wallets are exactly zero: the open obligation has drained. The mirrors record the contra: $w_{\text{DTC\_depot\_mirror}}[\text{GS}]$ is now -100 (GS has delivered the shares), and $w_{\text{JPMC\_nostro\_mirror}}[\text{GS}]$ is now $+5{,}000.00$ (GS has received the cash).

The real-wallet w_{us} is *not touched* by this transaction. The economic position recognised at T is preserved.

Conservation check at finality

Unit	Δw_{us}	$\Delta w_{\text{PS}}[\text{GS}, *]$	$\Delta w_{\text{DTC\_mirror}}$	$\Delta w_{\text{JPMC\_mirror}}$	Σ
XYZ	0	$+100$	-100	0	0
USD	0	$-5{,}000.00$	0	$+5{,}000.00$	0

Table 4: Conservation at finality: the per-unit sum across the constant universe is zero.

Wallet snapshot post-finality

```
w_us.own(XYZ)                       = +100
w_us.own(USD)                       = +995,000.00
w_PS[w_us, GS, XYZ].own(XYZ)        =  0          (drained)
w_PS[w_us, GS, USD].own(USD)        =  0          (drained)
w_DTC_depot_mirror[GS].own(XYZ)     = -100        (GS delivered out)
w_JPMC_nostro_mirror[GS].own(USD)   = +5,000.00   (GS received cash)
```

4.6 The four-state delta table

We collect the wallet states across the four temporal slices in a single table, in delta form (relative to pre-trade). The reading is: at every wall-clock t, the per-unit row sum is zero over the named slice of the constant universe. (The framework's absolute form of conservation is the inductive conclusion of the Conservation Lifting Theorem, Section 8.8; the per-state delta is the inductive step.)

Table 5: USD cash-leg deltas across the buy lifecycle. Pre-trade $w_{\text{us}}.\text{own}(\text{USD}) = 1{,}000{,}000.00$.

State	$w_{\text{us}}(\text{USD})$	$w_{\text{PS}}[\text{us}, \text{GS}, \text{USD}]$	$w_{\text{JPMC\_mirror}}[\text{GS}]$	Σ_{internal}
T^- pre-trade	0	0	0	0
T^+ post-TX1	$-5{,}000.00$	$+5{,}000.00$	0	0
$T + 1$	$-5{,}000.00$	$+5{,}000.00$	0	0
$T + 2^-$ pre-finality	$-5{,}000.00$	$+5{,}000.00$	0	0
$T + 2^+$ post-finality	$-5{,}000.00$	0	$+5{,}000.00$	0

Table 6: XYZ securities-leg deltas across the buy lifecycle.

State	$w_{\text{us}}(\text{XYZ})$	$w_{\text{PS}}[\text{us}, \text{GS}, \text{XYZ}]$	$w_{\text{DTC\_mirror}}[\text{GS}]$	Σ_{internal}
T^- pre-trade	0	0	0	0
T^+ post-TX1	+100	−100	0	0
$T+1$	+100	−100	0	0
$T+2^-$ pre-finality	+100	−100	0	0
$T+2^+$ post-finality	+100	0	−100	0

4.7 PnL at $T+2$ close: path independence

`XYZ` closes at \$51.50 on $T+2$. The portfolio's value:

$$V_{T+2} \;=\; 995{,}000.00 + 100 \cdot 51.50 \;=\; 1{,}000{,}150.00,$$

giving

$$\text{PnL}_{T+2} \;=\; -50.00, \qquad \text{PnL}_{\text{cum}} \;=\; +200.00 - 50.00 \;=\; +150.00.$$

The cumulative \$150.00 of P&L is computable in two completely different ways:

1. **Day-by-day:** +\$200 on $T+1$, −\$50 on $T+2$, sum +\$150.
2. **End-to-end:** $V_{T+2} - V_T = 1{,}000{,}150 - 1{,}000{,}000 = +\$150$.

The two computations agree by the v10.3 path-independence theorem (P10): cumulative PnL between two timestamps is a pure function of the wallet state and price vector at those timestamps, independent of the intermediate trading or settlement activity. The deferred-settlement extension preserves the property: the open-window contras flow through the cpty_virtual wallets without touching w_{us}.own, and the path-independence of the real-wallet PnL is unaffected.

4.8 What we have so far

After this section, the reader has seen:

- The full move sequence for an ordinary T+2 buy: one transaction at T, zero moves on $T+1$, zero moves on $T+2$ morning, one finality transaction on $T+2$ afternoon.
- Conservation verified at every state, in print, summing to zero on every row.
- P&L computed at T, $T+1$, and $T+2$, with explicit numerical agreement between the day-by-day and end-to-end calculations.
- The role of the three wallet classes: real holds the economic position, cpty_virtual holds the open-window contra, csd_virtual mirrors the external book at finality.
- The lifecycle FSM advancing from `Pending` (at T) to `Instructed` (state-only on $T+1$) to `Discharged` (witnessed at $T+2$).

The next section repeats the exercise for the standard sell, where the signs flip but the algebra is the same. After that, we add variants.

5 The Standard Sell

The sell is symmetric to the buy, modulo signs. The point of working it out explicitly is twofold: first, to verify that the canonical signed-sign convention of the cpty_virtual wallets handles both directions of trade with the same recon identity; second, because errors in sign discipline are the single most common defect in deferred-settlement implementations, and an independent verification on the sell catches them before they reach production.

5.1 Setup

- **Trade.** Sell 100 `XYZ` at $50.00; cash leg $5,000.00.
- **Execution time.** T = 2026-04-30 at 14:32:11 UTC.
- **Intended settlement.** t_d = 2026-05-04.
- **Pre-trade.** w_{us}.own(XYZ) = 100 (we have the security to deliver), w_{us}.own(USD) = 1,000,000.00.
- **Counterparty.** GS broker (same as before). Mirror wallets pre-trade flat.
- **Mark-to-market.** T: $50.00. $T+1$: $48.00. $T+2$: $47.50. (Falling prices favour the seller; we will see PnL in the right direction.)

5.2 The move block at T

```
Transaction TX1_SELL:
  type:    ECONOMIC_RECOGNITION
  tx_id:   hash_jcs(business_event_id="exec:XNYS:sell_id_77:leg_combined",
                    attempt_seq=0)
  ts:      2026-04-30T14:32:11Z

  -- Securities leg: economic recognition. We owe 100 XYZ to GS.
  Move 1:
    from = w_us
    to   = w_PS[w_us, GS, XYZ]
    unit = XYZ
    qty  = D_0(100)
        --> w_us:    +100 - 100 =    0
        --> w_PS:       0 + 100 = +100   (we owe GS 100 XYZ; payable side)

  -- Cash leg: economic recognition. GS owes us 5,000 USD.
  Move 2:
    from = w_PS[w_us, GS, USD]
    to   = w_us
    unit = USD
    qty  = D_2(5000.00)
        --> w_PS:       0 - 5,000 = -5,000  (negative = GS owes us 5,000 USD)
        --> w_us: +1,000,000 + 5,000 = +1,005,000

  L_15 register o_sec  (state = Pending; discharge_predicate =
    ByAttestation(sese.025))
  L_15 register o_cash (state = Pending; discharge_predicate =
    ByAttestation(camt.054))
```

Reading the moves

Move 1 sources 100 `XYZ` from w_{us} (where we held them) and credits $w_{\text{PS}}[w_{\text{us}}, \text{GS}, \text{XYZ}]$. The cpty_virtual wallet is now at +100. By the sign convention, a positive balance means *we owe*: we owe GS 100 `XYZ`. The trading book is now flat in `XYZ`.

Move 2 sources \$5,000.00 from $w_{\text{PS}}[w_{\text{us}}, \text{GS}, \text{USD}]$ (which was at zero, so the source goes negative) and credits w_{us}. The cpty_virtual wallet is now at −\$5,000.00. By the sign convention, a negative balance means *they owe us*: GS owes us \$5,000.00. The real wallet has \$1,005,000.00 (the cash receivable booked at trade time).

Conservation check at T

Unit	Δw_{us}	$\Delta w_{\text{PS}}[\text{XYZ}]$	$\Delta w_{\text{PS}}[\text{USD}]$	Δ csd mirrors	Σ
`XYZ`	−100	+100	0	0	0
`USD`	+5,000.00	0	−5,000.00	0	0

Table 7: Conservation at T on the sell.

5.3 PnL at $T+1$ close

The same canonical formula applies: V_t is the sum, over the firm's *real* wallets, of qty · price. The cpty_virtual contras do not enter valuation (Section 4.3).

$$\boxed{V_t \;=\; w_{\text{us}}.\text{own}(\text{USD}) \;+\; w_{\text{us}}.\text{own}(\text{XYZ}) \cdot P_t(\text{XYZ})}$$

The setup of Section 5 is a *long-to-flat* sale: pre-trade the firm holds $w_{\text{us}}.\text{own}(\text{XYZ}) = 100$ and sells those exact shares. Post-trade, $w_{\text{us}}.\text{own}(\text{XYZ}) = 0$ and the open obligation $w_{\text{PS}}[w_{\text{us}}, \text{GS}, \text{XYZ}] = +100$ is the contra to inventory already moved out of w_{us}. The firm has *no economic XYZ exposure* during the open window: the delivery obligation is met from inventory the firm has already segregated, at a price already locked at T. Subsequent price moves cannot affect the firm.

Substituting:

$$\begin{aligned} V_{\text{pre}} &= 1{,}000{,}000.00 + 100 \cdot 50.00 &&= 1{,}050{,}000.00, \\ V_T &= 1{,}005{,}000.00 + 0 \cdot 50.00 &&= 1{,}005{,}000.00, \\ V_{T+1} &= 1{,}005{,}000.00 + 0 \cdot 48.00 &&= 1{,}005{,}000.00, \\ V_{T+2} &= 1{,}005{,}000.00 + 0 \cdot 47.50 &&= 1{,}005{,}000.00. \end{aligned}$$

The drop from $V_{\text{pre}} = 1{,}050{,}000$ to $V_T = 1{,}005{,}000$ is the trade itself: the firm parted with 100 XYZ (worth \$5,000 at \$50) in exchange for a \$5,000 cash receivable, which has *not yet* been credited to w_{us} (it will arrive at finality via the JPMC mirror drain). The cash receivable lives in $w_{\text{PS}}[w_{\text{us}}, \text{GS}, \text{USD}] = -5{,}000$ during the open window and does not enter V_t until it lands on a real wallet at $T+2$.[1]

PnL at $T+1$.

$$\text{PnL}_{T+1} \;=\; V_{T+1} - V_T \;=\; 0.$$

The PnL is genuinely zero in the open window for a long-to-flat sale: the firm has no economic XYZ exposure post-execution, and the cash receivable is held at face. This is the correct symmetry with the buy: the buy creates economic exposure at T (the firm becomes long); the sell extinguishes economic exposure at T (the firm becomes flat). In the buy, mark moves in $[T, t_d]$ produce PnL; in this long-to-flat sell, they cannot, because there is nothing to mark.[2]

[1] This asymmetry — the cash leg of the sell sits on a cpty_virtual receivable while the cash leg of the buy decremented w_{us} at T — reflects the move-block discipline of Section 5: the cash leg is sourced from the cpty_virtual wallet (driving it negative) and credited to w_{us} only at finality. Section 8.1's recon identity reconciles the apparent gap in V_t against the open receivable; valuation does not double-count.

[2] A genuine *short sale* (firm flat or short pre-trade, selling 100 XYZ short) would carry −100 XYZ of economic

5.4 $T + 2$ close and finality

At \$47.50, $V_{T+2} = 1{,}005{,}000.00 + 0 \cdot 47.50 = 1{,}005{,}000.00$, so $\mathrm{PnL}_{T+2} = 0$ and $\mathrm{PnL}_{\mathrm{cum}} = 0$ over the open window. The price drop benefits no one in this trade: the seller has locked in \$5,000 of cash to receive at t_d; the buyer (GS or its client) has locked in 100 XYZ to receive at \$50 and absorbs the \$2.50 mark loss on its own books. The economic recognition occurred at T; the open window is purely operational.

The finality transaction is symmetric to the buy:

```
Transaction TX1_SELL_FINAL:
  type:    SETTLEMENT_FINALITY
  tx_id:   hash_jcs("FINAL", TX1_SELL.tx_id, sese.025_msg_id, camt.054_msg_id)

  -- Securities leg: drain the payable into the DTC mirror (we delivered).
  Move 1: from = w_PS[w_us, GS, XYZ], to = w_DTC_depot_mirror[GS], qty = 100 XYZ
        --> w_PS:    +100 - 100 =    0
        --> mirror:    0 + 100 = +100   (GS depot mirror long 100 XYZ)

  -- Cash leg: drain the receivable from the JPMC mirror (GS paid us).
  Move 2: from = w_JPMC_nostro_mirror[GS], to = w_PS[w_us, GS, USD], qty = 5,000
    USD
        --> mirror:   0 - 5,000 = -5,000   (GS nostro mirror short)
        --> w_PS: -5,000 + 5,000 =    0

  L_15 update o_sec.state:  Instructed -> Discharged
  L_15 update o_cash.state: Instructed -> Discharged
```

5.5 Symmetry verification

The buy and the sell run on the same machinery: same wallet classes, same FSM, same conservation algebra, same recon identity, same canonical valuation formula $V_t = \sum_{w\,\mathrm{real}} w.\mathrm{own}(u) \cdot P_t(u)$. The signs flip on the cpty_virtual balances; the moves' source and destination flip. PnL behaviour is dictated by the firm's residual real-wallet exposure, not by a buy-vs-sell branch: the buy leaves the firm long 100 XYZ during the open window (mark moves produce PnL); this long-to-flat sell leaves the firm with no XYZ exposure during the open window (PnL is zero by construction); a genuine short sale (Section 6) leaves the firm short and PnL signs flip with price direction. There is no special case, no buy-vs-sell branch in the framework. This is the test the design must pass: that a single piece of code, with no asymmetry between buy and sell, handles every case correctly. We verified it numerically in print; the property test that exercises it across thousands of random scenarios is described in Section 11.9.

exposure during the open window: that is a different setup, with its own active set, and is treated as a variant in Section 6 composed with the SBL borrowing leg.

6 Variants

The standard buy and sell of Sections 4 and 5 are the canonical case. Real markets present a long tail of variations: shorter or longer settlement cycles, partial fills, fails, restated finality, cross-border edge cases. The design is parameterised on the settlement window, and every variant we have catalogued reduces to a different value of that parameter or a different witness arrival pattern, never to a different architecture. This section walks the variants one at a time, each motivated by a case the simpler model does not cover.

6.1 T+1 markets

Since 28 May 2024, US cash equities settle T+1. The UK and EU are scheduled to follow on 11 October 2027. India has been at T+1 since 2023, with optional same-day settlement (T+0) for a subset of the National Stock Exchange list since 2024.

The Ledger absorbs T+1 because the intended settlement date t_d is a parameter on the settlement instruction, sourced from:

1. ProductTerms[u].settlement_cycle per ISIN $\times$ MIC, sourced from the calendar leaf L_4 and the reference master L_{16};
2. Trade.cdm_payload.settlementDate (CDM-native);
3. L_{15}.Obligation.deadline;
4. L_{18} break-aging timers.

No code change is required beyond a configuration refresh on the per-(MIC, ISIN) settlement-cycle table and a recalibration of the CSDR penalty window. The FSM is unchanged. Conservation is unchanged. The recon identity is unchanged. Every invariant in Section 12 holds with $t_d - T = 1\,\text{bd}$ exactly as it does with $t_d - T = 2\,\text{bd}$.

6.2 T+0 atomic settlement

In the limit $t_d \to T$, the open window collapses to zero. This is the regime of atomic distributed-ledger settlement: the trade and the custody movement coincide. The discharge predicate becomes `ByAttestation(on-chain finality oracle)`; the inflight period reduces to ε (the gap between the order submission and the on-chain confirmation block); the CSDR penalty regime does not apply (there are no fails); the FX leg becomes atomic PvP. There is no unsettled-trade capital under CRR Article 378–380; ECL on settlement receivables is approximately zero.

What stays constant:

- The lifecycle FSM (the obligation transitions `Pending` $\to$ `Discharged` in a single tick, but it does transition).
- The L_{15}.Obligation row (it exists for the duration of one block confirmation; downstream consumers — audit, regulatory, accounting — expect to see it).
- The conservation invariant.
- The trade-date economic recognition (it happens at T, even if $t_d = T$).
- The deterministic regulatory reporting projection.

This is the test of the architecture. A design that special-cases T+0 has not understood the deferred-settlement problem; the framework's design is that T+0 is a degenerate value of the settlement-cycle parameter, with the same machinery running with shorter timer durations.

6.3 Partial settlement

CSDs occasionally settle obligations in pieces. A 100-share buy may receive a `sese.025` for 60 shares on $T + 2$ and a second `sese.025` for the remaining 40 on $T + 4$. The framework handles this without special architecture.

```
Transaction TX_partial_1 (at T+2):
```

```
  type:    SETTLEMENT_FINALITY
  tx_id:   hash_jcs("FINAL", TX1.tx_id, sese.025_partial1_msg_id, attempt_seq=0)
  Move:    from = w_DTC_depot_mirror[GS], to = w_PS[w_us, GS, XYZ],
           unit = XYZ, qty = D_0(60)
  Move:    from = w_PS[w_us, GS, USD],     to = w_JPMC_nostro_mirror[GS],
           unit = USD, qty = D_2(3000.00)
  L_15 update o_sec.state:  Instructed -> PartiallySettled  (qty_settled=60,
    remaining=40)
  L_15 update o_cash.state: Instructed -> PartiallySettled  (qty_settled=3000,
    remaining=2000)

Transaction TX_partial_2 (at T+4):
  type:    SETTLEMENT_FINALITY
  tx_id:   hash_jcs("FINAL", TX1.tx_id, sese.025_partial2_msg_id, attempt_seq=1)
  Move:    same shape, qty = D_0(40) and D_2(2000.00)
  L_15 update both: PartiallySettled -> Discharged
```

Each partial step is conservation-preserving on its own quantity. The FSM transitions `Instructed` → `PartiallySettled` → `Discharged` monotonically. The recursion bound $D_{\max} = 2$ caps partial-then-buyin-then-partial cascades; beyond that, the obligation transitions to `Failed` → `Compensated` (cash-in-lieu) rather than continuing to fragment.

6.4 Failed settlement and the CSDR regime

A `Failed` state is a witness-driven transition. It is not a rollback. Two paths arrive at `Failed`:

1. A `sese.024` (settlement-failure-status advice) arrives with a normalised reason code mapped to `FailureReason`.
2. The deadline t_d + watchdog_window is reached with no positive witness; the watchdog emits a `silence_attestation` envelope and a four-eyes operator transitions the obligation to `Failed` with reason `DeadlineMissed`. Auto-fail by clock alone is forbidden (Section 11.3).

No moves on real wallets. The economic position recognised at T is preserved. The cpty_virtual wallet still carries the open contra. The csd_virtual mirrors are still flat. From the perspective of the trading book, the only thing that has changed is the obligation's lifecycle state and the CSDR penalty clock that begins accruing.

The CSDR penalty obligation is a separate L_{15}.Obligation row of kind `CSDR_PENALTY`, registered atomically with the failure transition.

```
Obligation row of kind CSDR_PENALTY:
  obligation_id        : hash_jcs(parent_obligation_id, failing_party_lei,
                                  accrual_date, schema_version)
  kind                 : CSDR_PENALTY
  discharge_predicate  : ByDeadline(buy_in_resolution_date)
  fields:
    parent_obligation_id : <parent o.obligation_id>
    failing_party_lei    : <LEI>            (party against whom this accrues)
    accrual_date         : <yyyy-mm-dd>     (one row per accrual day)
    qty_failed           : D_0 or D_18
    price_at_intended    : D_8 (pinned via refdata at intended settle date)
    rate_bps             : D_4 (per ESMA Penalty Matrix; pinned via refdata)
    accrual_start        : intended_settle_date + 1 bd
  state                : Pending
```

The penalty amount is a pure function of (qty_failed, price_at_intended, days_late, instrument_class, refda
No human inputs; no discretion. The rate-table is pinned via L_7^{P}.PolicyConfiguration versioning;
the daily accrual is emitted by the `CsdrPenaltyAccrualWorkflow` (Section 11.5).

The deterministic ID recipe is load-bearing: the inclusion of `failing_party_lei` ensures

that bilateral-DvP fails (where each party fails the other's leg) and chained fails on the same trade do not collide on obligation_id. The inclusion of `schema_version` ensures that an ESMA rate-table migration cannot silently re-issue penalty IDs. This recipe is canonical; the Ledger never overrides it; reconciliation against `semt.044` advice from the CSD uses the same recipe locally.

6.5 Reconciliation across the open window

Every business day in the open window admits a reconciliation against the external nostro and depot statements. The morning reconciliation engine emits one of three classifications per (w, u) pair:

1. `CLEAN` — the algebraic identity (Section 8) holds within tolerance.
2. `EXPECTED LEAD-LAG` — the identity is violated by an amount exactly explained by open cpty_virtual contras and open inflight wires. No action; this is the open-window state.
3. `BREAK` — the identity is violated by an amount not explained by (1) or (2). A row is created in the break register L_{18} with kind, owner, deadline, and expected resolution.

The classification is purely algebraic. There is no manual triage. We define the identity and demonstrate it holds on both the buy and the sell in Section 8.

6.6 Wire recall

After a payment has been wired and the obligation has been transitioned `Instructed` → `Discharged` on the cash leg, the counterparty's bank may issue a recall: a `camt.056` (FIToFI Payment Cancellation Request), or a `pacs.004` (Payment Return) that clears as a debit on our nostro. The discharge witness is not erased — the original `camt.054` envelope remains in L_{11} — but the obligation reverts.

```
WIRE_RECALL transaction:
  tx_id:   hash_jcs("recall", original_tx_id, recall_msg_id, attempt_seq=0)

  -- Anti-moves of the original finality transaction:
  Move 1: from = w_JPMC_nostro_mirror[GS], to = w_PS[w_us, GS, USD],
          unit = USD, qty = D_2(5000.00)
  -- (other leg, if relevant, similar)

  L_15 update o_cash.state:    Discharged -> Pending
  L_15 update audit_chain:     wire_recall(witness_lei, ts, reason_code)
  L_15 keep csd_finality_witness: <unchanged; original envelope preserved>
```

The semantics are bitemporal:

- `as_of(t_known(d))` — the moment of original discharge — returns `state = Discharged`.
- `as_of(t_known(r))` — the moment of recall — returns `state = Pending`.
- `with_corrections_through(t_known(r))` returns `state = Pending` with the audit chain populated.

The framework's discipline (Invariant DS7, Section 12) that failure does not reverse the real-wallet position is unaffected: the recall is a Discharged-to-Pending reversal grounded in a positive witness (the recall envelope itself), not an inferred failure. DS4 generalised: discharge requires positive witness; un-discharge requires positive un-discharge witness. A subsequent `camt.054` (a fresh payment attempt) is a new `Instructed` → `Discharged` cycle on the now-`Pending` obligation, with a fresh `attempt_seq`.

6.7 Restated finality

A CSD occasionally restates a previous `sese.025` — e.g., correcting a quantity that was wrong by one share due to a corporate-action timing edge. The framework treats the restated message

as a new obligation observation, not as an in-place edit:

1. The original `sese.025` envelope remains in L_{11} at its original $(t_{\text{obs}}, t_{\text{known}})$.
2. The new `sese.025` envelope is appended at fresh $(t_{\text{obs}}, t_{\text{known}})$.
3. A new L_{15}.Obligation row is registered at the fresh t_{known}, linked to the original by `parent_obligation_id`.
4. The new finality move pair has its own tx_id = hash_jcs("final", original_tx_id, restatement_msg_id, att 0).

This is the conservative (and only correct) reading: every restatement is a t_{known} update creating a new row; the original row is immutable and remains queryable via `as_of(original_t_known)`. The alternative reading — in-place t_{obs} correction with audit log only — was considered and rejected because it conflicts with replay determinism over the canonical $(t_{\text{obs}}, t_{\text{known}})$ bitemporal scan key.

7 Composition Cases

The variants of the previous section change the timing or witness of a single trade. The composition cases of this section are different: they layer the deferred-settlement design on top of (or alongside) other Ledger machinery — the GPM six-coordinate model for securities lending, corporate-action processing, cross-currency settlement, and block-and-allocation. The point is that the design is structurally orthogonal: the settlement FSM does not interact with the SBL coordinates, the corporate-action lifecycle does not interact with the settlement workflow, and the cross-currency case decomposes into two coordinated obligations rather than a special primitive.

7.1 Short sale, with composition to the SBL six-coordinate model

A short sale composes deferred settlement with securities borrowing. The GPM extends `PositionState` from a scalar own to a six-tuple $(\text{own}, \text{onloan}, \text{borr}, \text{coll\_post}, \text{coll\_recv}, \text{coll\_rehyp})$; the available inventory for delivery is the projection

$$\text{avail}(e, u) \;=\; \text{own}(e, u) - \text{onloan}(e, u) + \text{borr}(e, u).$$

A short sale of q shares of u at T settling $T + 2$ writes $\Delta\text{own}_{w_{\text{us}}}(u) = -q$ at T. If own $\geq q$, the reduction is clean. If own $< q$, own goes negative — and that is correct under SSR Article 12(1)(c) locate. The locate Unit and the avail projection enforce SSR/Reg-SHO at the smart-contract guard level, before the trade is admitted to the move stream.

The seller's delivery obligation lives in L_{15}.Obligation and on the cpty_virtual wallet, *not* on the GPM. The GPM and the deferred-settlement design are structurally orthogonal:

- The GPM's coordinates $(\text{own}, \text{onloan}, \text{borr}, \ldots)$ describe the firm's position relative to the universe of holdings; they record what we own, what we have lent out, what we have borrowed, and how the collateral has flowed.
- The deferred-settlement triple records the open custody movement against a specific counterparty for a specific trade, whose final destination will eventually be the appropriate GPM coordinate of the appropriate party at finality.
- There is no interaction. The SBL smart contract emits moves on the GPM coordinates following its own state-only writes; the settlement FSM emits moves on the cpty_virtual / csd_virtual axis on its own clock.

A recall during the open window is an independent obligation: the borrower's recall right is a contractual term in the SBL agreement, exercised by emitting a recall obligation on L_{15}, which is processed by the SBL settlement workflow on its own settlement cycle. If `avail` drops below the obligation quantity, a buy-in saga spawns. The point is that "recall during settlement window" is not a special case; it is the composition of two ordinary obligations.

7.2 Block-and-allocation

The asset-management dominant flow: a block trade executed on the venue, allocated post-trade across multiple sub-accounts. The framework handles this without architectural addition.

```
Block trade B at T:
  Buy 10,000 XYZ at $50 from broker GS, settling T+2.
  L_15 register o_block (state = Pending) on a virtual block wallet w_block_v.

Allocation event A at T+0+epsilon (via MT519 / sese.020):
  Allocate 4,000 to fund_F1 (w_F1)
  Allocate 3,500 to fund_F2 (w_F2)
  Allocate 2,500 to fund_F3 (w_F3)

  Atomic transaction TX_alloc:
```

```
Move 1: w_block_v -> w_F1, unit=XYZ, qty=4000
Move 2: w_F1 -> w_PS[w_F1, GS, USD], unit=USD, qty=200,000
(similar for F2, F3)
L_15: spawn o_F1_sec, o_F1_cash, o_F2_sec, o_F2_cash, o_F3_sec, o_F3_cash
L_15 update o_block: state = Compensated  (kind = ALLOCATION_SPLIT)
```

Composition properties:

- **Conservation across the allocation step.** The block obligation closes (`Compensated` by allocation-split kind) and 2N child obligations spawn, where N is the number of sub-accounts. Sum of quantity across child obligations equals the block quantity. The atomic move set in `TX_alloc` is balanced.
- **Independent settlement per child.** Each $o_{F_i,\text{sec}}$ and $o_{F_i,\text{cash}}$ proceeds through the FSM independently. Partial-fill at the CSD on the block flows to per-child partials via the same `attempt_seq` mechanism.
- **Audit trail.** The block-to-allocation chain is captured in L_{15}.parent_obligation_id.
- **Allocation correction.** A mis-allocation (wrong fund attribution) is corrected via four-eyes CORRECTION on the allocation transaction; the children are reversed as anti-moves; new children spawn under a new allocation. Both versions persist in the audit log.

7.3 Corporate action in the open window

Suppose the trade in Section 4 has T = Monday and t_d = Wednesday, and the issuer's record date for a dividend is Tuesday. By the trade-date doctrine, the buyer (us) is the economic owner of the dividend; by the CSD register, the seller (GS, who has not yet delivered) is the registered holder on the record date and will receive the dividend from the issuer. A *manufactured payment* from the seller to the buyer must flow.

The framework registers this as an obligation on L_{15} of kind `MANUFACTURED_PAYMENT`, atomically with the corporate-action processing transaction. The manufactured-payment obligation is independent of the underlying settlement obligation: the trade settles at t_d delivering the security, and the manufactured-payment settles separately, possibly netted into a periodic counterparty cash settlement. The bitemporal predicate record_date $\in (T, t_d]$ is evaluated at the time of corporate-action processing using the bitemporal L_{15} scan and the bitemporal record-date attestation.

7.4 Cross-currency settlement and Herstatt visibility

A cross-currency cash trade — buy EUR, sell USD — has two settlement legs that may settle in different currency systems with different settlement cycles and different finality oracles. CLS handles this for the major-pair set; non-CLS legs (most EM, some minor majors) carry residual Herstatt risk.

The framework registers two L_{15} rows linked by a parent obligation:

```
L_15 register o_fx (parent):
  kind               : SettlementInstructionDelivery
  discharge_predicate : (o_ccy1.state == Discharged) AND (o_ccy2.state ==
    Discharged)

L_15 register o_ccy1: SettlementInstructionDelivery for the USD leg
L_15 register o_ccy2: SettlementInstructionDelivery for the EUR leg
```

At every wall-clock t in the open window, exactly the legs whose state is non-terminal are visible as quantified Herstatt exposure, by an indexed scan over L_{15}. The framework does not eliminate Herstatt risk — the exposure is real — but it makes it observable, quantifies it, routes the compensation in case of fail, and audits the loss. The IFRS 7.B11 disclosure (aggregate

cross-currency open exposure, CLS-eligible split, largest single-counterparty Herstatt exposure) becomes a deterministic projection over L_{15}.

8 Reconciliation to the Nostro and Depot

Every business day, every (w, ccy) pair, every (w, ISIN) pair must satisfy an algebraic identity against the external custodian's statement. This section states the identity, demonstrates it on the buy and the sell, and gives the morning recon report shape.

8.1 The fundamental algebraic identity

For every (w, ccy) pair where w is a real wallet, with the cpty_virtual storage signed per the canonical convention (positive = we owe; negative = they owe us):

$$\text{nostro\_external}(w, \text{ccy}, t) \;=\; w_t.\text{own}(\text{ccy}) \;+\; \sum_{\text{cpty}} w_{\text{PS}}[w, \text{cpty}, \text{ccy}]_t \;-\; \text{inflight\_out}(w, \text{ccy}, t) \;+\; \text{inflight\_in}($$

The corresponding identity for securities, replacing nostro with depot:

$$\text{depot\_external}(w, s, t) \;=\; w_t.\text{own}(s) \;+\; \sum_{\text{cpty}} w_{\text{PS}}[w, \text{cpty}, s]_t \;-\; \text{depot\_out}(w, s, t) \;+\; \text{depot\_in}(w, s, t).$$

The signed sum subsumes the more verbose "$\sum$ payable $-$ $\sum$ receivable" form: when storage is signed, the algebra collapses. The disclosure-layer gross numbers are computed by projection (Section 3).

8.2 Inflight projections, named

`inflight_out` and `inflight_in` are projections, not stored fields:

```
inflight_out(w, u, t) = SUM o.amount
                        OVER  o IN L_15 WHERE
                              o.real_wallet == w
                          AND o.unit == u
                          AND o.state IN {Instructed, PartiallySettled}
                          AND o.side == payable
                          AND o.wire_dispatch_witness IS NOT NULL
                          AND o.csd_finality_witness  IS NULL

inflight_in(w, u, t)  = SUM o.amount
                        OVER  o IN L_15 WHERE
                              o.real_wallet == w
                          AND o.unit == u
                          AND o.state IN {Instructed, PartiallySettled}
                          AND o.side == receivable
                          AND o.cpty_dispatch_witness IS NOT NULL
                          AND o.csd_finality_witness  IS NULL
```

These are computed at recon time over the L_{15} table (closed-form scan; no replay over the move stream). The witness fields (`wire_dispatch_witness`, `cpty_dispatch_witness`, `csd_finality_witness`) are columns on the L_{15}.Obligation row populated by the envelope intake handler (Section 11.1).

8.3 Verification on the standard buy

We verify the identity on the standard buy at $T+2$ morning, before finality has been witnessed.

State of w_{us} at this instant:

- $w_{\text{us}}.\text{own}(\text{USD}) = 995{,}000.00$.
- $w_{\text{PS}}[w_{\text{us}}, \text{GS}, \text{USD}] = +5{,}000.00$ (we owe GS).

- Inflight: `sese.023` has been dispatched but the cash wire to GS has not been completed; we treat `wire_dispatch_witness` as not-yet-populated. Hence inflight_out = 0, inflight_in = 0.

External nostro per JPMC's `camt.053`: $1,000,000.00 (the cash has not yet left).
Substituting into the identity:

$$\text{LHS} \;=\; 1{,}000{,}000.00, \qquad \text{RHS} \;=\; 995{,}000.00 + 5{,}000.00 + 0 - 0 \;=\; 1{,}000{,}000.00. \quad \checkmark$$

The identity holds. The interpretation is that JPMC still shows our balance *fat* by $5,000 relative to our internal own, exactly because we owe $5,000 to GS that we have not yet paid. That gap is reconciled by the cpty_virtual wallet sitting on the books at +$5,000.

8.4 Verification on the standard sell

State of w_{us} at $T + 2$ morning, before finality, on the sell:

- w_{us}.own(USD) = 1,005,000.00 (the cash has been recognised at trade time; we have booked the receivable against our internal balance).
- $w_{\text{PS}}[w_{\text{us}}, \text{GS}, \text{USD}] = -5{,}000.00$ (GS owes us).
- Inflight: zero on both sides.

External nostro: $1,000,000.00 (the cash has not yet arrived).
Substituting:

$$\text{LHS} \;=\; 1{,}000{,}000.00, \qquad \text{RHS} \;=\; 1{,}005{,}000.00 + (-5{,}000.00) + 0 - 0 \;=\; 1{,}000{,}000.00. \quad \checkmark$$

The identity holds. Now JPMC shows our balance *lean* by $5,000 relative to our internal own, exactly because GS owes us $5,000 that has not yet arrived. The same identity, the same algebra, the same sign convention, both directions of trade. This is the property a property test must verify across thousands of randomly generated scenarios; we have just demonstrated the two endpoints in print.

8.5 Constant-time scan

The identity is constant-time per (w, ccy):

```
SELECT  w_real,
        ccy,
        SUM(balance) AS sum_signed_ps
FROM    position_state ps
JOIN    wallet_registry wr ON ps.wallet_id = wr.wallet_id
WHERE   wr.wallet_class = 'cpty_virtual'
  AND   wr.real_wallet  = :w
  AND   ps.unit         = :ccy
GROUP   BY w_real, ccy;
```

The `inflight_out`/`inflight_in` projections add a second scan over L_{15} filtered on state $\in$ {`Instructed`, `PartiallySettled`} and witness presence. Both scans are indexed (Section 11.8). A million open trades aggregate into approximately 10^3 wallet rows; the scan is bounded by wallet count, not transaction count.

8.6 Morning recon report shape

```
counterparty | ccy | bucket    | gross_payable | gross_receivable | net      |
    trades | aging
-------------+-----+-----------+---------------+------------------+----------+--------+------
GS           | USD | T+0       |     12,500.00 |         5,000.00 |  +7,500  |
     4  | 0bd
```

```
GS          | USD | T+1       |      5,000.00 |             0.00 |  +5,000  |
     1  | 1bd
GS          | USD | T+2_TODAY |  1,250,000.00 |       812,000.00 | +438,000 |
    47  | 2bd
GS          | USD | OVERDUE   |          0.00 |       150,000.00 | -150,000 |
     1  | 3bd  AGED
JPMC        | USD | T+0       |     80,000.00 |        80,000.00 |        0 |
    12  | 0bd
```

A per-currency aggregate row on top of that, exposing the identity:

```
real_wallet:               w_us
ccy:                       USD
nostro_external_balance:   1,234,567.89   (camt.053 from L_11)
own_internal:              1,229,567.89
+ Sum w_PS[us, *, USD]:        5,000.00
- inflight_out:                    0.00
+ inflight_in:                     0.00
                          ----------------
expected_nostro:           1,234,567.89
break:                             0.00
```

The buckets (T+0, T+1, T+2_TODAY, OVERDUE) match CSDR aging conventions. The aging column is in business days from intended settlement.

8.7 What breaks mean and what they do not mean

A break is a specific, narrow thing. It is a mismatch between the algebraic identity and the external statement that is *not* explained by an open obligation. It is not the same as:

- An open obligation in the window (this is `EXPECTED LEAD-LAG`, not a break).
- A failed settlement (this is a `Failed` obligation lifecycle state with a known cause).
- A late witness (this is a watchdog signal, opening `wf-confirm-break`, but the obligation is in a known state).

A break is a piece of money or a piece of stock that is on one record and not on the other, with no obligation on the books to explain it. It is rare in normal operations, and when it occurs, it is the highest-severity operational signal the framework emits. It opens a row in L_{18}.BreakRegister with the CSDR-aligned aging ladder: `Open` → `Investigating` → `Assigned` → `Aged-1` → `Aged-3` → `Aged-5` → `Escalated` → `At-Risk` → `Material` → `Closed-{Clean | Adj | Waived}`, with four-eyes required on `Closed-Waived`.

8.8 The Conservation Lifting Theorem

The recon identity is not asserted; it is a corollary of a more fundamental theorem, which we state and prove now.

Theorem 8.1 (Conservation Lifting)**.** *Let $\mathcal{W}$ be the constant universe (real ∪ cpty_virtual ∪ csd_virtual ∪ inception; Definition 3.4) and let Σ_t denote the set of transactions ever committed at time-points $\leq t$. Under the following hypotheses:*

H1 (move balance) *Every transaction in Σ_t is balanced per unit: for each $\tau \in \Sigma_t$ and each unit u, $\sum_{m \in \tau,\, m.\mathrm{unit}=u} \mathrm{signed\_delta}(m) = 0$.*

H2 (virtual sign correctness) *Every move whose source or destination is a cpty_virtual or csd_virtual wallet writes the signed quantity per the canonical convention (positive on $w_{\mathrm{PS}}[w, \mathrm{cpty}, u] \Leftrightarrow$ "we owe"; positive on $w_{\mathrm{DTC\_depot\_mirror}}[\mathrm{holder}] \Leftrightarrow$ "this holder is long at the CSD").*

H3 (state-only moves vacuous) *State-only transactions (zero-move transactions that update L_{15} alone) contribute zero to every wallet sum.*

H4 (universe constancy) *$\mathcal{W}$ is fixed across t: no wallet is removed; new wallets enter at a defined zero balance.*

H5 (CORRECTION respects H1) *Every CORRECTION transaction is itself balanced (anti-moves balance the original moves they reverse, plus any compensating moves).*

Then $\sum_{w \in \mathcal{W}} w_t(u) = 0$ for every unit u and every wall-clock t.

Proof. By induction on $|\Sigma_t|$.

Base case. $|\Sigma_t| = 0$: every wallet is at zero by H4. The sum is zero.

Inductive step. Assume $\sum_{w \in \mathcal{W}} w_t(u) = 0$ for every u. Let τ be the next transaction at $t' > t$. By H1, τ is per-unit balanced. By H2, virtual-wallet signs are consistent. By H3, a state-only τ contributes zero. By H5, a CORRECTION τ is balanced. Hence

$$\sum_{w \in \mathcal{W}} w_{t'}(u) \;=\; \sum_{w \in \mathcal{W}} w_t(u) + \sum_{m \in \tau,\, m.\mathrm{unit} = u} \mathrm{signed\_delta}(m) \;=\; 0 + 0 \;=\; 0.$$

□

What the theorem buys. The recon identity (Section 8.1) is a consequence of the theorem combined with the csd_virtual mirror semantics: the csd_virtual wallet's value is, by construction, the contra of the external nostro/depot at the corresponding holder's account. The open-window conservation invariant is no longer an independent assumption; it is restated as a corollary and proved here.

9 Accounting and Regulatory Footprint

The deferred-settlement design is opinionated about accounting: trade-date economic recognition is mandatory at the ledger primitive level. This section gives the standards grounding, the journal entries, the CSDR fails regime, the IFRS 9 ECL treatment, and the CRR Article 378–380 capital scaling.

9.1 Trade-date accounting is mandatory

For every cash-equity, listed-debt, listed-derivative, and equivalent regular-way trade, trade-date accounting is mandatory at the ledger primitive level. The move stream recognises the economic position at trade-execution timestamp T. Settlement-date accounting is not supported as a primitive; downstream consumers requiring it produce it as a reporting projection:

$$\text{settled\_position}(w, u, t) \;:=\; w.\text{own}(u, t) - \sum\{\, \text{signed\_qty}(o) : o \in \text{open\_obligations}(w, u, t) \,\}.$$

The standards grounding is:

- IFRS 9 paragraph B3.1.3 (regular-way exception) and B3.1.5 (settlement-date alternative for held-to-collect; not applicable to FVTPL).
- ASC 320-10-25-3 and ASC 326-20-30-2 (CECL).
- IAS 32 paragraph 42 (offsetting of financial assets and liabilities).
- IFRS 13 paragraph B5.1.2A (Day-1 P&L recognition).
- v10.3 §13.2 (Single-Coordinate Move Principle).

Why settlement-date cannot be primitive: CRR Article 325 (FRTB) and Article 274 (SA-CCR) compute capital on trade-date positions; recognising them only at t_d misstates capital. The IFRS 13 Day-1 P&L rule and the v10.3 P10 path-independence theorem also require the trade-date primitive.

9.2 Balance-sheet substantiation

For the standard buy of Section 4, the journal entries at each business day:

At T close (trade-date entry).

```
Dr Financial assets - FVTPL (XYZ)              5,000.00
   Cr Settlement payable to counterparty           5,000.00
```

At $T + 1$ close (mark to $52).

```
Dr Financial assets - FVTPL (XYZ)                200.00
   Cr Profit for the period (FVTPL)                 200.00
```

At $T + 2$ finality.

```
Dr Settlement payable to counterparty          5,000.00
   Cr Cash at custodian                            5,000.00
Dr Securities at custodian (sub-ledger)        5,200.00
   Cr Financial assets - FVTPL (XYZ, in transit)   5,200.00
```

No P&L impact at settlement. The reclass moves the position from the FVTPL-in-transit sub-ledger to the custodian sub-ledger; both are sub-classifications of the same balance-sheet line. Path-independence (v10.3 P10) holds across the cycle.

9.3 Five-document audit evidence chain

The framework's settlement record produces five documents that together form an end-to-end audit chain:

#	Document	Source	Framework artefact	Linked by
1	FIX 8=ExecRpt	Trading venue / OMS	`Transaction(SETTLEMENT)` in L_{13}	`external_ref`
2	`sese.023` out	Settlement → CSD	o_{sec}.state Pending→Instructed	`EndToEndId`
3	`sese.025` in	CSD → settlement	o_{sec}.csd_finality_witness	`RelatedRef`
4	`camt.054` in	Cash agent	o_{cash}.csd_finality_witness	`EndToEndId`
5	CSD depot statement	CSD direct feed	$w_{\text{DTC\_depot\_mirror}}[*]$ tie-out	position-level

Table 8: Five-document evidence chain. Each document is signature-verified and content-hashed at ingress per the attestation envelope (Section 11.1). Standards: ISA 500.6, ISA 505, PCAOB AS 2310, BCBS 239 P6, SOX 404 / SOC 1, IFRS 7.B11.

9.4 IFRS 9 ECL on settlement receivables

Settlement receivables are financial assets and bear expected-credit-loss. The standard's two-day-PD calibration depends on counterparty class:

Counterparty class	2-day PD	LGD	ECL on £100m	Treatment
CCP-cleared	< 0.1 bp	5–10%	$< £5\,000$	de minimis (IFRS 9.B5.5.30 p
Tier-1 dealer IG	0.5–2 bp	30–40%	£15k–£80k	rating-pooled
Sub-IG bilateral	5–20 bp	40–60%	£200k–£1.2m	track by name
Failed bilateral, day 5+	50–500 bp (Stage 2)	60–80%	£3m–£40m	Stage-2 migration

Table 9: IFRS 9 ECL on settlement receivables. CSDR fail past contractual date is a non-rebuttable Stage 2 trigger.

9.5 CRR Articles 378, 379, 380 (settlement risk capital)

- **Article 378** (Free deliveries): 100% risk weight until the second leg has taken place. Applies to free-of-payment (FoP) deliveries outside CSD-DvP.
- **Article 379** (Settlement risk, CRR III, in force 1 January 2025): risk-weight ramp on $\max(0, P_t - P_{\text{contract}}) \cdot \text{qty}$ as a function of business days past intended settlement: 100% (5–15 days), 625% (16–30), 937.5% (31–45), 1,250% (46+).
- **Article 380** (Large exposures): receivables exceeding 10% of CET1 trigger Article 395 reporting; 25% cap.

A worked Pillar 3 case: tier-1 dealer fails a €50m EU-listed equity buy, $P_t - P_{\text{contract}} = +€2$/share, qty = 1m shares. At T+5, RWA = €2m × 100% = €2m. Aged to T+16, RWA = €2m × 625% = €12.5m, which at a 12% capital ratio ties up €1.5m of CET1. Aged to T+46, RWA = €25m, tying up €3m of CET1 *on a single failed trade*. This is the regulatory pressure that makes T+1 and T+0 strategically attractive.

9.6 Definitive regulatory matrix

Table 10: Definitive regulatory matrix for deferred-settlement events.

Regime	Trigger	What MUST be emitted	Format	Cadence	Dedup key
MiFIR Art 26 / RTS 22	New trade in scope	Transaction report (ISIN, MIC, LEI, qty, price, ts)	RTS 22 XML via ARM	T+1 by 23:59 NCA local	`(reporting_lei, internal_tx_id, version)`

Regime	Trigger	What MUST be emitted	Format	Cadence	Dedup key
EMIR Refit Art 9	New OTC derivative; lifecycle event	Trade + lifecycle reports (UTI, UPI, LEI)	ISO 20022 `auth.030`	T+1	`(reporting_LEI, UTI, action_type, event_type, seq)`
CSDR Art 7	Settlement instruction; daily fail; monthly penalty	Daily fail register; `semt.044` advice; buy-in instruction	ISO 20022 `sese.024/025/027`, `semt.044`	Daily T+1ISD+1; monthly	`EndToEndId` out; `MessageId` in
SFTR Art 4	New SFT	Trade report (UTI, action, principals, collateral)	ISO 20022 `auth.052`	T+1 dual-sided	`(reporting_LEI, UTI, action_type, event_date)`
FINRA SLATE (Rule 6500)	New SBL or covered loan	Loan-level (CUSIP, qty, rate, collateral)	XML to FINRA	Same-day for new; T+1 mods	`(loan_id, event_seq)`
Reg SHO	Short sale; locate; FTD past T+3	Locate record; FTD report; threshold list	SEC EDGAR + FINRA	T (locate); EOD T+3 (FTD)	`(broker, CUSIP, trade_date, locate_id)`
Pillar 3 / BCBS 239	Quarterly; CRR Art 379 buckets	Failed-trade aggregate by aging (5/16/31/46+ days)	XBRL FINREP/-COREP	Q+45 cal days	`(reporting_lei, period_end, taxonomy_version)`
IFRS 9 / IAS 1 / IAS 32	Periodic; settlement-fail material	Notes — receivables, ECL, large exposures	iXBRL (ESEF)	Q / annual	`(entity_lei, period_end, taxonomy_version)`

Two non-negotiable design points: every dedup key is content-addressed; versioning is bitemporal. Restatements to a regulatory submission are deterministic projections.

9.7 T+1 transition

Architectural: parameter, not architecture. The t_d value lives in ProductTerms[u].settlement_cycle, Trade.cdm_payload.settlementDate, L_{15}.Obligation.deadline, and L_{18} aging timers. No code change beyond per-(MIC, ISIN) settlement-cycle table refresh and CSDR penalty-window recalibration. Operational machinery (FX cutoffs collapse by half; ETF NAV cycles; DTC night-cycle elimination; corporate-action cum/ex window collapse) must adapt; the architecture absorbs.

9.8 The CORRECTION transaction policy

Some events legitimately require unwinding ledger state: a trade booking error, a counterparty-disputed trade, a CSD-restated settled amount. The framework provides a CORRECTION transaction class with strict four-eyes preconditions:

```
Transaction(type=CORRECTION):
  references           : <original_tx_id>
  reason_code          : closed-sum: BOOKING_ERROR | CPTY_DISPUTE |
    CSD_RESTATEMENT |
                                     CROSS_CORRECTION | REG_ALREADY_REPORTED |
                                     CORRECTION_OF_CORRECTION | OTHER
  justification        : <free text, mandatory, >= 50 chars>
  requester_lei        : <LEI; primary requester>
```

```
approver_lei         : <LEI; MUST != requester>
approver_role        : closed-sum: OPERATIONS_HEAD | RISK_OFFICER | CFO_DELEGATE
approval_timestamp   : <UTC>

-- For CROSS_CORRECTION (correction touching > 1 trade):
affected_tx_ids      : <list of original_tx_ids>
scope_attestation    : <signed attestation, separate signing key from approver>

-- For REG_ALREADY_REPORTED:
reg_amendment_handler: closed-sum: MIFIR_AMEND | EMIR_AMEND | SFTR_AMEND
amend_reference      : <prior submission reference>

-- For CORRECTION_OF_CORRECTION:
prior_correction_tx_id      : <id of prior CORRECTION being corrected>
audit_committee_attestation : <required field; not skippable>
```

The framework rejects any CORRECTION where `requester_lei == approver_lei`, where `CROSS_CORRECTION` has < 2 affected_tx_ids, where `REG_ALREADY_REPORTED` is missing the amendment handler, or where `CORRECTION_OF_CORRECTION` is missing the audit-committee attestation. These are not procedural assertions; they are framework-level validation rules.

Bitemporal semantics. After a CORRECTION:

- Original transaction remains in L_{13} at original $(t_{\text{obs}}, t_{\text{known}})$.
- CORRECTION transaction appended at its own $(t_{\text{obs}}, t_{\text{known}})$.
- `as_of(t)` for t between original and correction returns original (un-corrected).
- `with_corrections_through(t_known')` returns corrected.

This is IAS 8.42 prior-period restatement discipline, structurally implemented via the bitemporal model.

10 CDM Mapping

The framework adopts the FINOS Common Domain Model (CDM 6.0.0) as a canonical vocabulary. CDM maps cleanly to roughly half of the deferred-settlement design; a quarter requires extension; a quarter is currently absent and motivates four pull-request-sized extensions to the open-source CDM repository. This section gives the cross-walk, the strategic gaps, and the functorial classification.

10.1 Cross-walk inventory

The 24-element inventory of deferred-settlement artefacts mapped against CDM 6.0.0 classifies into Direct (CDM is the wire and storage representation), Partial (CDM is the wire representation, framework storage extends), and Missing (framework-native; CDM extension proposed). Summary classification: Direct 11; Partial 6; Missing 7.

Table 11: Deferred-settlement cross-walk to CDM 6.0.0.

Framework artefact	**CDM status**	**CDM type / event**	**Notes / extension**
Trade execution at T	Direct	`ExecutionEvent`	—
Trade economic terms	Direct	`TradableProduct`	—
Trade-date timestamp	Direct	`Trade.tradeTime`	—
Settlement date	Direct	`settlementTerms.settlementDate`	
Counterparty LEI	Direct	`Party.partyId`	—
Venue MIC	Direct	`Trade.venue`	—
ISIN / unit	Direct	`Asset.identifier`	—
Quantity / price	Direct	`Quantity / Price`	—
DvP settlement	Direct	`Transfer (DvP)`	—
`sese.023` out	Direct	ISO 20022 synonym	—
`sese.025` in	Direct	ISO 20022 synonym	—
Lifecycle Pending / Instructed / Settled	Partial	`TransferStatus`	enrichment needed (Gap 6)
Partial settlement	Partial	`Transfer.transferStatus`	granularity (Gap 7, subsumed in Gap 6)
`camt.054` in	Partial	ISO 20022 cash leg	link to CDM `Transfer`
Manufactured payment	Partial	`ManufacturedPayment` (rough)	per-jurisdiction rules pending
`sese.024` fail status	Partial	ISO 20022 synonym	failure-reason mapping owed (Gap 6)
CSDR cash penalty	Missing	—	extension PR-2 (Gap 8)
CSDR buy-in	Missing	—	extension PR-3 (Gap 9)
`Obligation` as first-class	Missing	—	extension PR-4 (Gap 10, highest leverage)
Economic-exposure-at-T	Missing	—	doctrinal (Gap 11)
Per-counterparty open-window	Missing	—	framework-native, no CDM equivalent
`cpty_virtual` wallet axis	Missing	—	lossy projection in F (§10.3)
Bitemporal restatement of finality	Missing	—	CDM is not bitemporal
`silence_attestation`	Missing	—	framework-native

10.2 Five strategic gaps

Gap 6 — `TransferStatus` enrichment

Add `Failed`, `PartiallySettled`, `Cancelled` states and a `TransferFailureReason` enumeration. CSDR-blocking. Strategic.

Gap 7 — Partial settlement granularity
Subsumed in Gap 6.

Gap 8 — CSDR cash penalty
Add a structural event `CSDRPenaltyDetail` with the framework's deterministic ID recipe.

Gap 9 — Buy-in event
Add `BuyInInstruction` and `BuyInExecution` types.

Gap 10 — Obligation as first-class root type
Highest-leverage. Adding a top-level `Obligation` type would let the framework's L_{15}.Obligation row map directly to CDM, eliminating the lossy projection in Gap 11.

The four PRs are non-breaking; the recommended sequence is PR-1 (Gap 6+7, ∼80 lines), PR-2 (Gap 8, ∼120 lines), PR-3 (Gap 9, ∼150 lines), PR-4 (Gap 10, ∼200 lines).

10.3 The forgetful functor F is lossy and non-faithful

Let **Lg** be the category of Ledger states (objects: (WalletRegistry, UnitRegistry, MoveStream, ObligationStore); morphisms: balanced atomic transactions). Let **CDM** be the category of CDM states (objects: collections of `BusinessEvent`, `TradeState`, `TransferState`; morphisms: `BusinessEvent` chaining).

Proposition 10.1 (F is a lossy non-faithful functor)**.** *The forgetful map $F : \mathbf{Lg} \to \mathbf{CDM}$ is a functor (it respects identities and composition: $F(\mathrm{id}) = \mathrm{id}$ and $F(\tau_1 \circ \tau_2) = F(\tau_1) \circ F(\tau_2)$), but it is neither injective nor faithful:*

- ***Lossy.*** *Distinct* **Lg***-states can map to the same* **CDM***-state. Specifically, F collapses the wallet axis: per-(wallet, unit)* `PositionState` *density is lost; only per-trade* `TransferState` *remains.*
- ***Non-faithful.*** *Distinct* **Lg***-morphisms can map to the same* **CDM***-morphism. Two distinct events in the move stream — a counterparty-leg move and a give-up-leg move on the same (w_{real}, u) pair — are distinct ledger morphisms (different* `tx_id`*, different cpty_lei on the cpty_virtual axis) but F projects them to the same CDM* `BusinessEvent` *because CDM lacks the cpty_virtual axis to distinguish them.*

10.4 The economic quotient

Define an equivalence relation $\sim$ on **Lg** states: $\sigma_1 \sim \sigma_2$ iff they project to the same per-trade economic position table (collapsing the wallet axis: sum across all cpty_virtual and csd_virtual wallets per (w_{real}, u)). The quotient $\mathbf{Lg}_{\text{econ}} = \mathbf{Lg}/\sim$ has objects = equivalence classes and morphisms = induced maps.

Proposition 10.2. *On $\mathbf{Lg}_{\text{econ}}$, F restricts to a homomorphism: $F|_{\mathbf{Lg}_{\text{econ}}}$ is faithful and injective up to CDM* `BusinessEvent` *identity.*

The direction of authority is unchanged: Ledger first, CDM downstream. The Ledger reconciles to CDM via $F|_{\mathbf{Lg}_{\text{econ}}}$, not the other way. After PR-4 (Gap 10 first-class `Obligation`), the quotient becomes finer (less collapsing), and F becomes faithful on more of **Lg**.

10.5 First-class unit representation: the bijection Φ

Section 3 mentioned that the cpty_virtual wallet design and an alternative "first-class unit" representation are information-equivalent. The equivalence is a bijection Φ that we state for completeness; the design choice between the two is operational, not mathematical.

Let M be the mainstream representation (cpty_virtual wallets, L_{15} obligations, transaction-level status as projection). Let C be the first-class representation: every obligation $o \in L_{15}$ gives rise to a fresh unit u_o°, the contra-quantity for o lives in PositionState$[w_{\text{real}}, u_o^\circ]$, and the lifecycle state lives in UnitStatus$[u_o^\circ]$. Then there is a bijection $\Phi : M \to C$ with explicit inverse Φ^{-1} such that $\Phi^{-1} \circ \Phi = \text{id}_M$ and $\Phi \circ \Phi^{-1} = \text{id}_C$. Conservation, recon identity, lifecycle FSM, and tx_status projection all transfer between the two representations componentwise. The mainstream is chosen because it requires no inflation of the unit master (which would otherwise grow from $O(10^4)$ to $O(10^7)$) and no opt-out at every `PositionState` consumer. When CDM 7.0 ships an `Obligation` root type (PR-4), Φ becomes the documented mapping the framework implements.

10.6 ISO 20022 messages

The framework's witness substream is ISO 20022 messages, mapped to internal events by versioned schema mappers. Each message is a witness to a state transition; it is not itself the transition.

Message	Direction	Effect
`sese.023`	out	dispatch settlement instruction; transitions Pending → Instructed
`sese.024`	in	settlement-failure-status advice; transitions to Failed
`sese.025`	in	settlement confirmation; transitions to Discharged or PartiallySettled
`sese.027`	out	cancellation request
`camt.053`	in	daily cash-balance statement; recon input
`camt.054`	in	cash credit/debit advice; cash-leg discharge witness
`camt.056`	in	FIToFI Payment Cancellation Request (wire recall)
`pacs.004`	in	Payment Return (cleared recall)
`semt.044`	in	CSDR penalty advice; recon input for penalty obligations
MT 54x (legacy)	in	SWIFT MT precursor to `sese.025/sese.024`

Table 12: ISO 20022 message synonyms used by the framework.

10.7 What CDM 6.0.0 features NOT to use

A handful of CDM features are present but not load-bearing for the deferred-settlement design, and using them would conflict with the framework's invariants:

- `partialCashSettlement` (CDS-only).
- `sixtyBusinessDaySettlementCap` (CDS auction).
- Settlement-date position recognition (violates Invariant DS1).
- Reversing position on fail (violates Invariant DS7).
- Deprecated `Lineage` type.

11 Implementation Notes

The formal specification states what the system must produce. This section gives the developer everything the formal specification does not say: the attestation envelope's signature scheme, the Temporal workflow shapes, the deterministic transaction-ID recipe, the type-level boundary, the storage and retention strategy, and the test cases the implementation must pass.

11.1 The attestation envelope

Every external observation that drives a state transition on L_{15} is wrapped in an `Envelope`.

```
Envelope:
  payload          : bytes (JCS-canonical CBOR or JSON of the message)
  payload_schema   : (schema_uri, schema_version)
  source_lei       : LEI of the signing entity
  signature        : Ed25519 signature over hash_jcs(payload || schema_version ||
    source_lei)
  ts_obs           : observation timestamp asserted by source (Z, ISO 8601)
  ts_known         : system-known timestamp at intake (assigned at L_11 arrival)
  dedup_key        : hash_jcs(schema_version, source_lei, ts_obs, payload)
  attestation_kind : { sese.025 | camt.054 | sese.024 | camt.053 |
                       affirmation | corp_action_ex_date | manual_override |
                       silence_attestation | wire_recall | ... }
```

The discipline is:

- **JCS canonicalisation (RFC 8785).** Every payload is canonicalised before hashing or signing. This kills equivocation attacks where reordering object keys would change the hash but preserve semantic content.
- **Ed25519 (RFC 8032)** signature, verified at L_{11} ingest by the public key registered for `source_lei` in L_7^{P}.TrustRoots (versioned, with CRL/OCSP check at verification time).
- **Schema pinning.** `payload_schema` references an immutable L_7^{P}.SchemaRegistry entry. ISO 20022 dialect divergence between SWIFT versions is contained at this column.
- **Idempotency.** Two envelopes with identical `dedup_key` are dropped post-arrival; the second arrival is recorded for audit but does not transition L_{15}.
- **Schema-version in dedup key.** Without the `schema_version` component, a schema migration would silently allow re-discharge with a structurally-different payload that hashes to the same key. This is the construction that makes Invariant DS19 (witness-identity determinism) hold.

11.2 Multi-source aggregation

When the CSD is silent, the framework admits discharge via a multi-source quorum. The rules are deliberately strict: the CSD is the primary source, and it is never substituted by a single non-primary source.

11.3 Absence-of-finality: silence is itself attested

Silence past t_d+watchdog_window does *not* auto-FAIL the obligation. It triggers a `wf-confirm-break` workflow:

```
Watchdog timer triggers when:
  now() > intended_settlement_date + Lambda_n
  AND  o.state in {Pending, Instructed, PartiallySettled}

On trigger:
  Spawn break-handler workflow wf-confirm-break(o.obligation_id).
  Emit a "silence-attested" envelope with:
```

Witness configuration	**Effect on L_{15}.state**
`Envelope(sese.025, source=CSD)` valid sig, schema, dedup-pass	Discharge admitted; FSM Instructed → Discharged
CSD silent past t_d + 0.5 bd; `Envelope(camt.054, source=custodian)` *and* `Envelope(affirmation, source=cpty)` both valid	Discharge admitted via 2-of-2 quorum
Single `Envelope(camt.054, source=custodian)` only	**Not admitted.** Logged. Watchdog continues.
Single `Envelope(affirmation, source=cpty)` only	**Not admitted.**
Two attestations contradicting each other (e.g., `sese.025` + `sese.024` for same `tx_id`)	**Quarantine via wf-confirm-break.** No discharge. Manual review.

Table 13: Multi-source aggregation policy. Quorum tightening to 3-of-3 (CSD + custodian + cpty) is configurable per (venue_mic, ccy_class) in L_7^{P}.QuorumPolicy; never relaxable below 2-of-2 by configuration.

```
  attestation_kind = silence_attestation
  source_lei       = our_lei  (we are attesting our own observation of silence)
  payload          = { obligation_id, intended_settlement_date,
                       observed_at, watchdog_seq=n }
Append to L_18.BreakRegister.

Do NOT transition L_15.state to Failed. Wait for human or further evidence.
```

The poll cadence Λ_n: $\Lambda_0 = 0$ (immediate at deadline), $\Lambda_1 = 30\,\mathrm{min}$, $\Lambda_2 = 2\,\mathrm{h}$, Λ_3 = end-of-day, $\Lambda_4 = +1\,\mathrm{bd}$. After Λ_4 with no resolution, ops escalate per the regulatory clock (CSDR penalties begin accruing on the failing party from $t_d + 1\,\mathrm{bd}$ regardless of Ledger state).

11.4 Transaction-ID recipe

```
tx_id = hash_jcs(business_event_id, attempt_seq)

where:
  business_event_id = stable namespaced upstream identity, e.g.:
                      "exec:" || venue_mic || ":" || exec_id || ":" || leg_label
                      "final:" || referenced_tx_id || ":" || msg_id_set
                      "buyin:" || referenced_tx_id || ":" || buyin_seq
                      "correction:" || referenced_tx_id || ":" || correction_seq
  attempt_seq = monotonic uint32 within (business_event_id);
                starts at 0, increments on workflow-level retry
```

Idempotency. Two workflow attempts producing the same `(business_event_id, attempt_seq)` produce the same `tx_id`. Two attempts at different `attempt_seq` produce different `tx_id`s; the move-stream side (L_{13}.tx_id is a primary key) prevents double-application. The `attempt_seq` is producer-monotonic, generated by the writing workflow, and carried across Temporal `ContinueAsNew` boundaries via the workflow-input payload — never via Temporal's ephemeral `run_id`.

11.5 Temporal workflow architecture

Workflows are per-obligation, not per-trade or per-bucket: workflow_id = `"settlement-saga:"` || obligation_id
A DvP trade with two legs has two L_{15} rows and two workflow instances. They communicate via signals when atomicity at discharge is required.

Workflow	Trigger	Body (high-level)
`SettlementSaga`	trade-time obligation creation	emit `sese.023`; await finality witness or deadline+watchdog; on witness atomically apply finality moves + L_{15} transition; on deadline+watchdog spawn `wf-confirm-break`; on restated finality `ContinueAsNew` with new `attempt_seq`
`BuyInWorkflow`	o.state = Failed past CSDR buyin_window	emit buy-in instruction; await execution attestation; emit replacement trade with own `tx_id`; on completion, signal parent `SettlementSaga` which transitions parent $o \to$ BoughtIn
`MorningReconWorkflow`	Temporal cron 09:00 local	for each (w, u) scan recon identity; classify CLEAN / EXPECTED LEAD-LAG / BREAK; emit reports; on BREAK, spawn `wf-position-break`
`CsdrPenaltyAccrualWorkflow`	spawned by `SettlementSaga` on $\to$ Failed transition	daily timer at CSD-local EOD; on tick, emit a CSDR_PENALTY L_{15} row per accrual day; terminate when parent $\to$ terminal
`OutageWatchdogWorkflow`	Temporal cron, 1-min tick per csd_lei	probe primary CSD ingest channel; if unresponsive $> T_{\text{outage\_grace}}$, broadcast `csd_outage_detected`; open L_{18} break; await OUTAGE_RESUME
`RestatementWatchWorkflow`	inbound `sese.025` with `original_msg_id` populated	verify the original referenced exists; treat the new message as a new obligation observation; new L_{15} row at fresh t_{known}; original immutable

Table 14: The six Temporal workflows. `SettlementSaga` is the central one; the others handle specific lifecycle branches.

11.6 Signal-handler commutativity

Replay determinism (Invariant DS5) requires that any two interleavings of the same multiset of inbound signals produce the same final state. Most signal pairs commute; a few non-commuting cases are deterministic precedences encoded in the FSM step function.

The non-commuting cases are not violations of replay determinism; they are deterministic precedences (terminal absorbs late events; contradictions always quarantine; correction-vs-discharge resolves by precedence). A property test verifies the multiset-replay form of DS5 against this table.

11.7 Type-level boundary: phantom wallet classes

The single most leveraged type-discipline investment is to make wallet-class swaps unrepresentable at the type level.

```
type real_wallet
type cpty_virtual_wallet
type csd_virtual_wallet

val of_real : Wallet_id.t -> real_wallet wallet_handle
val of_cpty : Wallet_id.t -> cpty_virtual_wallet wallet_handle
val of_csd  : Wallet_id.t -> csd_virtual_wallet wallet_handle

val emit_trade :
```

Signal A	Signal B	Result	Commute?
`sese.025` for o_sec	`camt.054` for o_cash	both Discharged	yes
two partials (same tx_id)	—	accumulated by attempt_seq	yes
`sese.025`	`sese.027` cancel	terminal absorbs late events	**no** (precedence)
silence_attestation	`sese.025`	break opens, witness closes it	yes
`sese.024` fail	`sese.025` discharge	contradiction → quarantine	**no** (precedence)
correction	`sese.025`	terminal absorbs	**no** (precedence)
cross-trade signals	—	per-workflow isolation	yes

```
    real_wallet      wallet_handle
  -> cpty_virtual_wallet wallet_handle
  -> security : (Isin.t * Qty.t)
  -> cash     : (Currency.t * Cash.t)
  -> trade_date : TradeDate.t
  -> settle_convention : SettleConvention.t
  -> (Transaction.t * PairedObligation.t, trade_error) Result.t

val emit_discharge :
    PairedObligation.t
  -> cpty_virtual_wallet wallet_handle
  -> csd_virtual_wallet  wallet_handle
  -> csd_confirmation : Sese_025_msg.t
  -> (Transaction.t, discharge_error) Result.t
```

`emit_discharge` cannot be passed a `real_wallet wallet_handle`. A settlement-status mutation that touches the trading book's own coordinate is structurally unrepresentable. This enforces Invariants DS1 and DS17 at compile time.

A `PairedObligation` is constructed only via a smart constructor:

```
module PairedObligation : sig
  type t  (* abstract; constructed only via [pair] *)

  type pairing_error =
    | Different_trade_id     of Trade_id.t * Trade_id.t
    | Different_settle_dates of SettleDate.t * SettleDate.t
    | Mirrored_qty_mismatch  of { sec_qty : Qty.t; cash_qty : Cash.t;
                                  expected_cash : Cash.t }
    | Same_side_pairing
    | Wrong_leg_units        of { sec_unit : Unit_id.t; cash_unit : Unit_id.t }

  val pair :
      security_leg : Obligation.t
    -> cash_leg     : Obligation.t
    -> (t, pairing_error) Result.t

  val security_leg : t -> Obligation.t
  val cash_leg     : t -> Obligation.t
end
```

There is no public function from a single `Obligation.t` to a `DischargeResult.t`. Discharge consumes a `PairedObligation` atomically; the DvP-E (economic-recognition) atomicity

becomes a property of the type system, not a procedural assertion.

11.8 Storage and retention

A typical tier-1 dealer processes 10^7 trades per day with two legs per trade. Over seven years of regulatory retention this is approximately 5×10^{10} obligation rows, or roughly 10–15 TB of hot+warm storage including envelope payloads.

Tier	Window	Storage class / SLA
Hot	0–90 days	OLTP (Postgres / DynamoDB), < 100 ms p99
Warm	90 days – 7 years	Columnar (Iceberg / Parquet on S3), < 5 s ad-hoc
Cold	7 years – 25+ years (regulatory archive)	Glacier or equivalent, hours-to-days

Table 15: Retention tiering. Bitemporal queries on hot+warm; cold-tier queries materialise via async restore.

Partitioning:

```
L_15.Obligation:  PARTITION BY (year_month(intended_settlement_date),
                                hash_prefix_8(business_event_id))
L_13.MoveStream:  PARTITION BY (year_month(t_obs),
                                hash_prefix_8(tx_id))
L_11.ExternalConfirmation: PARTITION BY year_month(t_known)
```

Hash-prefix-8 sharding gives 256 shards per month — supports 10^8 rows per shard which is below typical OLTP node capacity.

Indexes:

- (counterparty_lei, lifecycle_state) B-tree — ops queues.
- (wallet_id, unit, lifecycle_state) hash — recon identity.
- (intended_settlement_date, lifecycle_state) B-tree — settlement-day windows.
- (business_event_id, attempt_seq) unique B-tree — idempotent intake.
- `dedup_key` on L_{11} — envelope dedup.

11.9 Test cases the implementation must pass

The walking-skeleton test set is the sign-off prerequisite at every release boundary. All twelve must pass.

Table 16: Walking-skeleton test set.

Test	Scenario	Verification
WS-1	BUY happy (T+2)	§4 reproduced; conservation tables sum to zero; recon identity holds
WS-2	SELL happy (T+2)	§5 reproduced; recon identity verified independently
WS-3	T+1 happy	T+1 cycle parameter; FSM unchanged
WS-4	T+0 atomic happy	T+0 parameter; inflight ε
WS-5	recon-lag E2E	recon stable across t_{obs} lag up to 1 bd cross-border
WS-6	short happy	locate-converted-to-borrow; SBL composition
WS-7	recall in window	recall obligation independent of sale; buy-in saga on insufficient avail
WS-8	cum/ex CA in window	manufactured-payment obligation registered

Test	Scenario	Verification
WS-9	FX Herstatt	cross-currency obligation pair; one-leg-settled state visible
WS-10	partial settlement	$D_{\max} = 2$ chain; per-step conservation
WS-11a	DvP CSD-reject SYMMETRIC	both legs Failed; no real-wallet move
WS-11b	DvP CSD-reject ASYMMETRIC	one leg Discharged, other Failed; `tx_status = Mixed`; `wf-confirm-break`
WS-12	manual override	four-eyes preconditions; FSM Pending $\rightarrow$ Compensated

In addition, a property-based test framework with hand-authored generators (not record-and-replay from production traces) exercises every closed sum from the FSM and every numbered invariant. A reference interpreter — a single-threaded, single-machine, deterministic-clock pure-functional implementation $\sim$2 kLOC — serves as a differential oracle: any property test that fails on the production runtime but passes on the reference (or vice versa) localises the bug to one of the two. The reference is the canonical specification.

12 Invariants DS1–DS19

The deferred-settlement extension introduces twelve primary invariants and seven restated v10.3 invariants, totalling nineteen labelled DS1 through DS19. The numbering is preserved from the specification source; gaps in the sequence (DS2, DS5, DS6, DS8, DS13–DS16) correspond to invariants that are restated v10.3 properties or were folded into other invariants during the design's review history. We give the twelve primary invariants here as numbered `invariant` blocks; the appendix to this section lists the restated v10.3 invariants by reference.

Note on numbering. The numbering DS1, DS3, DS4, DS7, ... reflects pruning during R3 review — DS2, DS5, DS6, DS8, and DS13–DS16 were restated v10.3 invariants moved to the appendix, and the gaps are deliberate.

Invariant 12.1 (DS1 — Economic-Exposure-at-T)**.** *For all transactions τ, all real wallets w that are a party to τ, all units u, and all wall-clock times $t \in [T_{\text{exec}}(\tau), \infty)$, the change in* $w.\text{own}(u)$ *from immediately before T_{exec} to time t is exactly the sum of signed deltas on w for unit u across the moves of τ recorded by that time. Equivalently: there is no projection over* $(\text{PositionState}[w_{\text{real}}, u].\text{own})$ *alone whose value during $[T, t_d]$ differs from its value after τ has reached* `Discharged`*, holding the price function constant. Projections over the cpty_ virtual wallet family* are *permitted to vary during the open window; they are precisely the* inflight_in *and* inflight_out *named projections. Type: hybrid (compile-time phantom wallet class + runtime property test). Severity: CRITICAL.*

Invariant 12.2 (DS3 — Reconciliation Identity)**.** *For every (w, ccy) pair with w real, the algebraic identity of Section 8.1 holds at every wall-clock t within the per-currency tolerance pinned in L_7^{P}. The corresponding identity for securities holds for every (w, ISIN) pair. Type: runtime. Severity: HIGH.*

Invariant 12.3 (DS4 — No Discharge Without Witness)**.** *For every L_{15}.Obligation row o in any state other than* `Pending` *or* `Cancelled`*, there exists a verified envelope ε recorded on L_{11} such that* $\text{verify}(\varepsilon)$ *holds (signature, schema, dedup, multi-source quorum where required) and ε matches o's discharge or fail predicate. The framework never marks a leg* `Discharged` *by inference, by clock, or by absence of evidence. Type: hybrid (FSM consumes an envelope as parameter at compile time; cryptographic verify at runtime). Severity: CRITICAL.*

Invariant 12.4 (DS7 — Failure Non-Reversal)**.** *For every transition of any L_{15}.Obligation row to* `Failed`*, no real-wallet* own *coordinate changes as a consequence. The economic position recognised at T is preserved across failure. The only legal exception is a CORRECTION transaction with explicit anti-moves and four-eyes approval. Type: hybrid (capability typing + mutation testing). Severity: CRITICAL.*

Invariant 12.5 (DS9 — Buy-In Compensation Closure)**.** *For every obligation o in state* `Failed` *for duration at least the CSDR buy-in window, there exists a corresponding buy-in transaction τ_{buyin} and a buy-in obligation o_{buyin} such that o.*compensation_handler *has been executed, ending the chain in* `BoughtIn` *or* `Compensated`*. Type: runtime (Temporal saga + property test). Severity: HIGH.*

Invariant 12.6 (DS10 — Cross-Currency Herstatt Visibility)**.** *For every cross-currency transaction, the framework registers a parent obligation o_{fx} with two child obligations $o_{\text{ccy1}}, o_{\text{ccy2}}$, each with its own settlement date and discharge witness. The parent's discharge predicate is the conjunction of the children's discharge. At every t, exactly the open obligations whose state is non-terminal are visible as quantified Herstatt exposure. Type: runtime (projection). Severity: HIGH.*

Invariant 12.7 (DS11a — Partial-Settlement Per-Step Conservation)**.** *For every partial-settlement event at attempt sequence n: with q'_n delivered at attempt n and $q_n - q'_n$ remaining, $q_{n+1}^{\text{remaining}} +$*

$q'_n = q_n$ *and* $q_{n+1}^{\text{remaining}} \geq 0$. *Per-step conservation across the partial-fill chain. Type: runtime. Severity: HIGH.*

Invariant 12.8 (DS11b — Partial-Settlement Monotonicity)**.** *For every obligation entering* `PartiallySettled`*: (i) its state advances monotonically (no return to* `Pending`*); (ii) the economic position on the real wallet is unchanged by partial-settle events (the original transaction recognised the full quantity at* T*); (iii) the partial chain terminates within* $D_{\max} = 2$ *partial-buyin cycles, beyond which the obligation transitions to* `Failed` $\rightarrow$ `Compensated` *(cash-in-lieu). Type: runtime. Severity: HIGH.*

Invariant 12.9 (DS12 — Variant Degeneration)**.** *The deferred-settlement representation is parameterised on* settle_date $= t_d$ *and degenerates uniformly across T+0 / T+1 / T+2 / T+5+. For all variants, DS1, DS3, DS4, DS7, DS9, DS10, DS11a, DS11b, DS17, DS18, DS19 hold without modification; only workflow timer durations change. Type: structural (proof by reduction over* t_d*). Severity: MEDIUM.*

Invariant 12.10 (DS17 — Capability Scoping on Settlement Status Writes)**.** *The writer capability for* o.state *transitions on every* L_{15} *row is held by exactly one workflow class per row. Different rows may have different writer classes. Specifically:* `SettlementSaga` *writes the original obligation row;* `BuyInWorkflow` *writes a separate buy-in obligation row;* `CsdrPenaltyAccrualWorkflow` *writes its own* `CSDR_PENALTY` *rows;* `CorrectionTransactionService` *performs CORRECTION-class transitions at the move-stream-level write capability. No other handler may mutate any* o.state *outside its scoped row class. Type: compile-time (phantom typing on writer capability). Severity: HIGH.*

Invariant 12.11 (DS18 — DvP Ledger-Level Atomicity)**.** *For every transaction* τ *containing both a securities leg and a cash leg in the same* L_{13} *move pair, the executor commits both legs atomically: both apply or neither. Type: structural (executor primitive). Severity: CRITICAL.*

Invariant 12.12 (DS19 — Witness-Identity Determinism)**.** *For every envelope* ε *with payload* P *signed by source LEI* L *at observation timestamp* t *with schema version* v*,* $K(v, P, L, t) =$ hash_jcs(v, L, t, P) *is unique and deterministic; two envelopes are identical iff their dedup keys are equal. The forward implication (*K *equal* $\Rightarrow$ *semantically identical) is enforced by JCS canonicalisation plus collision-resistance of the hash; the reverse is enforced by determinism of* `hash_jcs` *over a totally-ordered argument list. Type: hybrid (compile-time total function over typed envelope schema; runtime collision-check at intake). Severity: BLOCKING. (Without DS19, DS4 cannot be enforced.)*

Type-vs-runtime decomposition

Restated v10.3 invariants

The following v10.3 invariants are restated in the deferred-settlement context but not reasserted as new:

- Conservation $\sum_{w \in \mathcal{W}} w_t(u) = 0$ (DS2) — corollary of the Conservation Lifting Theorem, Section 8.8.
- Replay determinism (DS5) — restated; explicit commutativity table in Section 11.5.
- Idempotency of finality messages (DS6) — restated; envelope dedup_key in Section 11.1.
- Status monotonicity (DS8) — property of the FSM closed sum.
- CSDR penalty schema determinism (DS14) — pure function over L_7^{P}.
- Counterparty default close-out (DS15) — restatement of v10.3 close-out routing.
- Bitemporal restatement (DS16) — structural property of the bitemporal model.

Invariant	**Type**	**Mechanism**	**Severity**
DS1	hybrid	phantom wallet class + property test	CRITICAL
DS3	runtime	property-test on BUY and SELL	HIGH
DS4	hybrid	FSM takes envelope as parameter; verify	CRITICAL
DS7	hybrid	capability typing + mutation testing	CRITICAL
DS9	runtime	Temporal saga + property test	HIGH
DS10	runtime	projection over L_{15}	HIGH
DS11a	runtime	property-test on residual conservation	HIGH
DS11b	runtime	property-test on monotonicity + $D_{\max}$	HIGH
DS12	structural	same FSM under t_d parameter	MEDIUM
DS17	compile-time	phantom typing on writer capability	HIGH
DS18	structural	executor primitive	CRITICAL
DS19	hybrid	`hash_jcs` total + collision-check	BLOCKING

Table 17: Invariant decomposition by enforcement mechanism.

13 Out of Scope

The framework is opinionated about what it does not address. The following are deliberately excluded from v11.0 scope, with reasons.

Liquidity management of the cash leg
The Ledger's responsibility ends at the L_{15} obligation. The treasury system is a downstream consumer that schedules, prioritises, and funds the wires. The Ledger's role is to make the obligations observable, ageing, and ranked; the treasury role is to act on them. We refuse to fold treasury policy into the ledger because the policy is firm-specific, jurisdiction-specific, and time-varying, while the ledger record is canonical.

Title vs beneficial ownership
The legal layer is outside the framework. Whether a particular CSD-recorded position constitutes legal title or merely beneficial ownership depends on the master agreement, the CSD rulebook, and local insolvency law. The framework records the economic and custody facts; the legal characterisation is a downstream attestation by the firm's counsel.

Cross-CCP novation during the open window
When a trade is novated from a bilateral booking to a CCP-cleared booking during the settlement window, the framework supports it as a CORRECTION transaction with explicit anti-moves and a fresh trade booking. The novation policy itself — when it occurs, who signs it, what fee structure applies — is governed by the CCP rulebook and is not a primitive in the framework.

Mass-cancellation events
Per-trade CORRECTION with four-eyes is supported. Bulk cancellation (an entire trading desk, an entire counterparty pair, an entire venue) is a workflow on top of the per-trade primitive, not a primitive of its own. Implementing it as a primitive would create a single privileged path that bypasses the per-trade four-eyes discipline; we refuse.

Smart-constructor 14-rejection cases
An ideal smart constructor for `Obligation.create` would reject 14 distinct misuse patterns at construction (qty ≤ 0; FoP with cash; settle date in past; cash/qty mismatch; etc.). This is valuable hardening but requires coordinated rollout across upstream OMS, allocation engine, and corporate-action service. Deferred to v12 RFP.

Phantom-typed accounting basis
Distinguishing trade-date-basis and settlement-date-basis accounting in the type system would let downstream consumers express their basis preference without ambiguity. The current framework commits to trade-date-only at the primitive level, so the phantom split is not load-bearing yet. Deferred.

First-class unit u° representation
The bijection Φ (Section 10.5) makes this representation information-equivalent to the mainstream cpty_virtual design. Operational cost (unit-master inflation, every-consumer opt-out) makes mainstream the v11.0 choice. Deferred to v12 RFP for re-evaluation when CDM 7.0 ships an `Obligation` root type.

Per-jurisdiction manufactured-payment rules
The structural slot is reserved as L_{15}.Obligation kind `MANUFACTURED_PAYMENT`. The per-jurisdiction tax-residency rule pin (US, EU, UK, JP) is owed work, depending on tax-attorney input.

Per-CSD failure-reason normalised mapping
The closed-sum `FailureReason` type exists at the type level. The per-CSD enum mapping table for DTC, Euroclear, Clearstream, T2S is operational rollout work; deferred from the formal specification.

A note on what is *not* out of scope, since earlier drafts of this design listed it as such: **CSD operational outage**. The framework now defines a `OutageWatchdogWorkflow` (Section 11.5) and an OUTAGE_RESUME correction class. Multi-day outages (T2S March 2023; DTC night-cycle 2024) are first-class operational events with a defined freeze-then-resume protocol.

14 Glossary

Attestation envelope
A structured wrapper around an external observation: payload, schema, source LEI, Ed25519 signature, JCS-canonicalised, with observation and known timestamps and a content-addressed dedup key. The framework's only admissible source of evidence for a state transition. Defined in Section 11.1.

`attempt_seq`
Producer-monotonic uint32 carried across Temporal `ContinueAsNew` via the workflow input payload, used in the `tx_id` formula to make retries idempotent.

`business_event_id`
Stable namespaced identifier of an upstream event (e.g. `exec:XNYS:exec_id_42:leg_combined`). Half of the input to `tx_id`.

CDM
FINOS Common Domain Model; the canonical machine-executable vocabulary the Ledger adopts.

`cpty_virtual`
Wallet class. A per-counterparty virtual wallet keyed $(w_{\text{real}}, \text{cpty\_lei}, u)$ holding the open-window contra-quantity for that real wallet against that counterparty for that unit. Sign carries the side.

CSDR
Central Securities Depositories Regulation. The EU regime that governs settlement discipline, including cash penalties on fails (Article 7) and mandatory buy-in.

`csd_virtual`
Wallet class. A mirror wallet representing the position of an external holder at a CSD or correspondent bank, keyed $(\text{csd\_lei}, \text{holder\_lei}, u)$. Absorbs the contra at finality.

`dedup_key`
$\text{hash\_jcs}(\text{schema\_version}, \text{source\_lei}, \text{ts\_obs}, \text{payload})$. The content-addressed identifier that makes envelope intake idempotent and witness identity deterministic (Invariant DS19).

Discharge predicate
The condition under which an obligation transitions to `Discharged`. For most cash-equity obligations it is `ByAttestation(Envelope, sese.025|camt.054, ref=tx_id)`.

$D_{\max}$
Recursion bound on partial-settlement chains. Pinned at 2; beyond, the obligation transitions to `Failed` $\rightarrow$ `Compensated`.

DvP
Delivery versus Payment. The CSD's discipline that securities and cash legs of a trade settle atomically at the settlement utility. The framework distinguishes DvP-L (ledger atomicity, our move pair), DvP-S (settlement-utility atomicity, the CSD's batch), and DvP-E (economic-recognition atomicity, the `PairedObligation` type).

Envelope
See *attestation envelope*.

F The forgetful functor from **Lg** to **CDM**. Lossy and non-faithful; restricts to a homomorphism on the economic quotient $\mathbf{Lg}_{\text{econ}}$.

Finality
The CSD's irreversible commitment that a settlement movement has occurred. Witnessed by an ISO 20022 `sese.025` on the securities leg and `camt.054` on the cash leg.

Herstatt risk
The cross-currency settlement risk that arises when one leg of an FX trade has settled and the other has not. The 1974 failure of Bankhaus Herstatt is the canonical case; CLS handles it for the major-pair set.

`hash_jcs`
A hash computed over JCS-canonicalised input (RFC 8785). Used as a content-addressed identifier in `tx_id`, `dedup_key`, and CSDR penalty obligation IDs.

Inception move
A v10.3 §2.7 move at system genesis time that establishes the absolute baseline. Pre-trade balances are the result of inception moves; the framework's deltas sit on top of that baseline.

Inflight
A projection over L_{15} rows that are `Instructed` or `PartiallySettled` with witness fields in defined states. Not a stored field. Used in the recon identity.

ISD
Intended Settlement Date. The date t_d at which finality is contractually due.

L_n
Leaf in the data layer's master taxonomy. The framework references in particular: L_4 (calendar conventions), L_7^{P} (policy configuration including trust roots, schema registry, mapper registry, quorum policy), L_{11} (external confirmations / envelopes), L_{13} (move stream), L_{15} (obligations), L_{16} (reference master), L_{17} (regulatory submissions), L_{18} (break register), L_{19} (clock authority).

Lifecycle FSM
The finite state machine on L_{15}.Obligation.state. Closed sum of eight states with a defined transition relation. Section 3.6.

Mirror wallet
A `csd_virtual` wallet that mirrors an external holder's position at a CSD or correspondent bank. Drains the cpty_virtual contra at finality.

Multi-source quorum
Discharge admitted via 2-of-2 from (custodian, cpty) when the primary CSD is silent past $t_d + 0.5$ bd. Tightenable to 3-of-3; never relaxable below 2-of-2.

`PairedObligation`
An abstract type whose only constructor is the `pair` smart constructor. Encodes DvP-E atomicity at the type level: `emit_discharge` consumes a `PairedObligation`, not two separate obligations.

`PS` / `PSS`
Surface forms of cpty_virtual wallets: `w_PS[w, cpty, ccy]` for cash-leg, `w_PSS[w, cpty, ISIN]` for securities-leg. Both are instances of class `cpty_virtual`.

`real`
Wallet class. A book-of-record wallet whose `own` coordinate represents an economic position the firm holds. Written at T by `apply_trade_move`; never moved by settlement.

Recon identity
The algebraic equality between the external nostro (or depot) balance and the internal own plus signed cpty_virtual sum plus inflight projections. Section 8.1.

`SettlementSaga`
The Temporal workflow per obligation. Emits the `sese.023`, awaits the witness or watchdog, applies finality moves, transitions the obligation. Section 11.5.

`silence_attestation`
An envelope with `attestation_kind = silence_attestation` emitted by the watchdog when the deadline passes with no positive witness. Recorded on L_{11}, opens `wf-confirm-break`, but does *not* transition the underlying obligation.

StatesHome
The v10.3 principle that all state lives in exactly one of three maps (`ProductTerms`, `UnitStatus`, `PositionState`) plus the `WalletRegistry`. The deferred-settlement extension touches only `PositionState` and `WalletRegistry` sidecar.

t_d Intended settlement date. A parameter on the trade and on L_{15}.Obligation.

t_{obs} / t_{known}
The two clocks of the bitemporal model. `ts_obs` is the observation timestamp asserted by the source; `ts_known` is the system-known timestamp at intake.

Trade-date accounting
Recognising the economic position at the trade-execution timestamp T, not at t_d. Mandatory at the framework's primitive level. Standards: IFRS 9.B3.1.3, ASC 320-10-25-3.

tx_id
hash_jcs(business_event_id, attempt_seq). Content-addressed transaction identifier. Section 11.4.

`tx_status(tx)`
The pure projection function over the per-leg L_{15} rows of a transaction, returning the trade-level status. Not a stored field. Section 3.7.

Watchdog
A Temporal cron + heartbeat timer in `SettlementSaga` that emits a `silence_attestation` envelope at deadline-passing and spawns `wf-confirm-break`. Never auto-fails an obligation.

Witness
A verified attestation envelope that establishes a state transition. Discharge requires positive witness; un-discharge (e.g. wire recall) requires positive un-discharge witness.

Closing

The deferred-settlement design is a parameter, not a doctrine. The same machinery — two new wallet classes, one obligation row, one workflow — handles T+0 atomic settlement, T+1 cash equities, T+2 EU equities, and T+5 cross-border fails with no architectural change. The economic position is recognised at T on the trading book; the custody movement is recognised at t_d on the mirror; the open-window contra sits on a per-counterparty virtual wallet between them; and conservation holds, by construction, over the constant universe of all four wallet classes (real, cpty_virtual, csd_virtual, inception).

The reader who built this design move-by-move from Section 4 now has a working mental model. Every numerical claim in the document was computed in print. Every conservation table summed to zero. Every variant was added because the simpler model could not handle a case the variant exposes. The result is that an engineer can write the code from this document and an auditor can check the result against the same document; both will be working from the same canonical source.

Document version. v1.0, April 2026. Companion to the Ledger Framework Specification v10.3 and to the data-layer specification v1.0. Sole authorial voice: R. Delloye.

References

[1] R. Delloye, "The Ledger: A Closed-System Framework for Financial Settlement — Unified Design Specification: Architecture, Lifecycle, and CDM Integration," v10.3, March 2026.

[2] R. Delloye, "The Ledger Data Layer: Static, Reference, Market, Oracle, Smart-Contract Execution, and Listed-Instrument Data: A Specification," v1.0, April 2026.

[3] R. Delloye, "The Ledger Valuation Stack: Pricing, Greeks, Calibration, PnL Attribution, and Temporal Orchestration," v1.0, March 2026.